Information Fusion 124 (2025) 103345



Contents lists available at ScienceDirect

Information Fusion

journal homepage: www.elsevier.com/locate/inffus



Full length article



# Byzantine-robust federated learning considering distributed Non-IID data in privacy-sensitive scenarios

Zongpu Wei, Jinsong Wang *, Zening Zhao, Zhao Zhao

*School of Computer Science and Engineering, Tianjin University of technology, Tianjin, 300384, China*
*Tianjin Key Laboratory of Intelligence Computing and Novel Software Technology, Tianjin, 300384, China*
*National Engineering Laboratory for Computer Virus Prevention and Control Technology, Tianjin, 300457, China*

ARTICLE INFO

Dataset link: https://github.com/zzzzzzp02/ProFL

*Keywords:*
Federated learning
Byzantine attack
Privacy preserving
Defense strategy
Linearly homomorphic encryption

ABSTRACT

For federated learning (FL), its decentralized training approach is vulnerable to the targeted or untargeted attacks from Byzantine clients, which results in decreased model accuracy. In privacy-sensitive scenarios, the Byzantine clients identification is extremely difficult, due to the fact that gradient information is encrypted. To preserve privacy while identifying Byzantine clients in these scenarios, many schemes on identifying malicious local gradients over ciphertexts are proposed. However, existing works suffer from the problems of inefficiency in non-independent identically distributed (Non-IID) data and potential privacy leakage. To address the issues, we propose a privacy-preserving and byzantine-robust FL scheme (ProFL), which can resist encrypted malicious gradients effectively and protect the privacy of gradients. In ProFL, the linearly homomorphic encryption (LHE) technique is adopted to safeguard the gradient parameters. Specifically, ProFL designs two secure protocols that effectively mitigate potential threats in existing methods while maintaining computation complexity within a linear range. A robust aggregation method based on Manhattan distance is proposed to effectively enhance the robustness of federated aggregation in Non-IID data scenarios. The security and convergence proof of ProFL is given. Finally, extensive evaluations demonstrate that ProFL outperforms other privacy-preserving and byzantine-robust FL schemes in defending against various poisoning attacks.

## 1. Introduction

Federated learning (FL) is a collaborative machine learning framework where a global model in the center server is trained by multiple local clients without sharing their data. It effectively strikes a balance between model training and data privacy protection. Currently, FL has emerged as a pivotal technology for handling sensitive tasks such as those in the Internet of Things (IoT) [1,2], autonomous driving [3], etc.

In the FL, the training and uploading of local model parameters are entirely completed by clients, in which each client is expected to comply with the protocol and train the model properly. Some clients may send malicious gradient information to the central server, which results in contamination of the global model [4,5] or a significant decrease in its accuracy [6,7]. These clients are called Byzantine clients and the uploaded malicious gradients are called poisoned gradients. Taking Fig. 1 as an illustration, the "green line" represents the benign gradient, and the "yellow line" and "red line" represent the poisoned gradients. The purpose of Byzantine clients typically falls into two distinct categories. The first category involves targeted attacks [4], where the global model is manipulated to produce specific target labels for a given sample. The second category encompasses untargeted attacks [7], which involve indiscriminate flipping or perturbation of the gradient.

Unintentional leakage of sensitive data can occur through the transparency of gradient information in traditional FL schemes. For instance, model inversion attacks [8,9] can effectively reconstruct original training set sample features from gradient information, and membership inference attacks [10] can determine if a given sample has been used in the training set based on the model output. For privacy-sensitive FL scenarios (e.g., internet of vehicles [3], smart medical [11], etc.), to eliminate the threat of privacy leakage, researchers employ various privacy-preserving techniques to protect gradient information such as homomorphic encryption (HE) [12,13], secure multi-party computation (SMPC) [14,15], trusted execution environments (TEE) [16,17], and differential privacy (DP) [18,19]. However, the adoption of these techniques makes it more difficult to identify Byzantine clients in privacy-sensitive FL scenarios because gradient information is encrypted.

* Corresponding author at: School of Computer Science and Engineering, Tianjin University of technology, Tianjin, 300384, China.
*E-mail address:* jswang@tjut.edu.cn (J. Wang).

https://doi.org/10.1016/j.inffus.2025.103345
Received 26 February 2025; Received in revised form 5 May 2025; Accepted 15 May 2025
Available online 2 June 2025
1566-2535/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

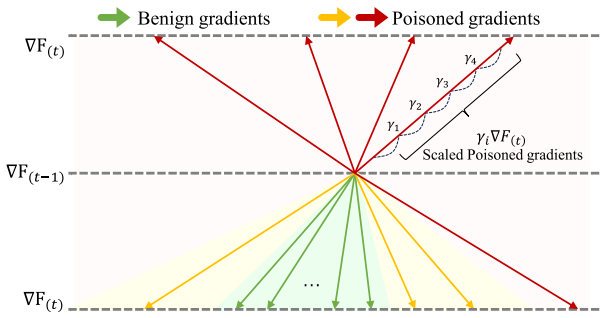



**Fig. 1.** Example of poisoned gradients in FL. Let $\gamma_i \nabla F_{(t)}$ denotes the poisoned gradient, which can be a vector of arbitrary length. $\nabla F_{(t)}$ denotes the gradient of $t$th iteration.

To address the dual challenges of privacy leakage and Byzantine attacks in such scenarios, some privacy-preserving and byzantine-robust FL schemes are proposed. These schemes design their schemes to identify clients with large deviations. For instance, gradient perturbations are introduced in [20–23] to protect the original gradients and detect similarity based on the perturbed gradients. However, these existing schemes expose the "shifted" distribution of the complete gradient [24]. This is a clear privacy violation [8,25], as it leaks more information, and does not meet the design goal of "privacy" claimed in their works. Furthermore, several privacy-preserving and byzantine-robust FL schemes [20,21,26] are exclusively applicable to the independently and identically distributed (IID) data. These schemes filter suspected poisoned gradients, which may incorrectly penalize benign gradients with significant deviations, resulting in their inapplicability to the non-independent identically distributed (Non-IID) data [22].

Based on the above observations, this paper proposes a privacy-preserving and byzantine-robust FL scheme called ProFL to identify Byzantine clients under stringent privacy protection. ProFL ensures data privacy by using linearly homomorphic encryption (LHE) technique to prevent local gradient leaks. First, to address the potential privacy leakage caused by the "shifted" distribution, a shuffling-based secure median protocol (**SSecMed**) is proposed to effectively conceal the parameter distribution as indistinguishable random numbers while ensuring the accuracy of the computational results. Furthermore, similarity metrics such as Pearson correlation coefficient [20], cosine similarity [22,23] and Euclidean distance [27] entail non-linear computational operations, rendering the implementation of these measures in the LHE cryptosystem challenging. Therefore, we propose a secure Manhattan distance (**SecMHD**) protocol that can calculate the Manhattan distance as metrics between any two gradients with linear complexity. Finally, to address the issue that existing solutions might wrongly penalize benign gradients with significant deviations, a novel aggregation weight is proposed to effectively resist Byzantine attacks and adapt to Non-IID data. The main contributions are summarized as follows.

- We propose ProFL, a privacy-preserving and byzantine-robust FL scheme combining LHE technique. In both IID and Non-IID data, ProFL exhibits remarkable capability in identifying Byzantine clients to ensure the robustness of FL system. Moreover, ProFL can safeguard data privacy during training by defending against attackers from snooping the local data of clients.
- ProFL designs two security protocols, specifically **SSecMed** and **SecMHD**, aimed at facilitating privacy-preserving defense and robust aggregation. ProFL supports both IID and Non-IID data and mitigates an arbitrary proportion (less than 50%) of Byzantine clients.
- We demonstrate extensive experiments on three real-world datasets to verify the performance of ProFL scheme. The results show that our ProFL scheme surpasses existing schemes in terms of two types of Byzantine attacks: targeted and untargeted attacks.

The remainder of this paper is organized as follows. In Section 2, we introduce the related literature. Section 3 gives the preliminary knowledge about typical FL and HE. Section 4 describes the system model, design goals, and so on. Our ProFL framework and methodology are proposed respectively in Section 5 and Section 6. Section 7 gives the security and convergence analysis. The performance of ProFL is evaluated in Section 8. Section 9 analyzes the strengths and weaknesses of ProFL and discusses its potential applications. Finally, Section 10 concludes this work.

## 2. Related work

In this section, we review related works on byzantine-robust FL and privacy-preserving byzantine-robust FL.

### 2.1. Byzantine-robust federated learning

The decentralized training approach of FL renders each client within the system untrustworthy. The client can locally manipulate the gradient information uploaded to the FL server [28]. Hence, many byzantine-robust FL schemes were proposed to reduce the negative effect of Byzantine clients and maintain the model accuracy. For instance, Blanchard et al. [29] first proposed a byzantine-robust FL scheme, named Krum, which aggregated a global model by selecting a set of local models with small differences in distance and averaging them together. Chen et al. [30] proposed a robust aggregation method, named coordinate-wise Median and Trimmed Mean, which sorted the values of all local gradients in a coordinate and then selected the median or trimmed mean of parameters as the global model. Cao et al. [31] assumed that the server had a clean root dataset and proposed a byzantine-robust FL scheme called FLTrust, which identified outliers by measuring the cosine similarity between each local gradient and the well-behaved model trained on the root dataset. Although these schemes filtered gradients that are suspected to be poisoned, they may mistakenly penalize benign gradients with significant deviations, rendering them unsuitable for the Non-IID data.

To achieve byzantine-robust FL in the setting of Non-IID data, some works personalized the weight adjustments of different local models during aggregation instead of filtering suspicious local models [32]. For instance, Xu et al. [32] proposed a defense mechanism called MinVar, which did not detect and eliminate outlier local updates from the global model aggregation. Instead, it took all local updates and assigned different weights to them in the global model aggregation. Peng et al. [33] employed a resampling strategy and geometric median to mitigate the influence of inner and outer variations in client models on aggregation. Zhai et al. [34] introduced a robust FL framework named BRCA through credibility assessment, which computed the credibility scores of each model update to detect Byzantine attacks in Non-IID data. However, these methods typically operate on plaintext parameters and do not safeguard the privacy of parameters. Unbearable costs or ineffective defenses arose when these defense methods were combined with existing privacy protection technologies (such as HE [12] or SMPC [14,15]).

### 2.2. Privacy-preserving and Byzantine-robust FL

To simultaneously address the requirements of privacy protection and the detection of Byzantine clients, some studies have proposed privacy-preserving and byzantine-robust FL. With respect to privacy, Liu et al. [20] proposed privacy-enhanced FL (PEFL) for detecting

**Table 1**
A comparative summary of existing byzantine-robust FL.

| Scheme | Setting | Defense Strategy | Privacy |
|---|---|---|---|
| Krum/MKrum [29] | IID | Euclidean distance | ○ |
| Median [30] | IID | Coordinate statistics | ○ |
| FLTrust [31] | IID | Cosine similarity Validation data | ○ |
| MinVar [32] | Both | Variance minimization | ○ |
| Var-reduced FL [33] | Both | Geometric median | ○ |
| BRCA [34] | Both | Credibility assessment | ○ |
| PEFL [20] | IID | Pearson correlation | ◐ |
| PBFL [21] | IID | Cosine similarity Validation data | ◐ |
| P2BroFL [26] | IID | Cosine similarity | ◐ |
| ShieldFL [22] | Both | Cosine similarity | ◐ |
| SEAR [27] | IID | Euclidean distance Coordinate statistics | ● |
| Our work | Both | Manhattan distance | ● |

*Notes:* ○denotes the scheme does not consider privacy. ◐denotes the scheme considers privacy but exposes the "shifted" distribution. ●denotes the scheme considers privacy without exposing the "shifted" distribution.

Byzantine attacks in IID data, which deployed the Pearson correlation coefficient and coordinate-wise medians to detect deviations from clients. Miao et al. [21] designed a privacy-preserving byzantine-robust FL (PBFL) scheme based on blockchain to mitigate the impact of the central server and malicious clients. PBFL applied cosine similarity to assess the difference between the trusted model trained on the root dataset and local models, thus filtering out any poisoned gradients uploaded by malicious clients. Ma et al. [22] proposed a privacy-preserving FL framework named ShieldFL, which introduced a secure cosine similarity method to measure the distance between two encrypted gradients. ShieldFL achieved robustness for both IID and Non-IID data. Ma et al. [35] proposed a differentially private byzantine-robust FL scheme (DPBFL) with high computation and communication efficiency. DPBFL designed a novel aggregation protocol based on the shuffle model. Zhao et al. [27] proposed a secure and efficient aggregation framework (SEAR) for byzantine-robust FL based on the TEE. Dong et al. [26] proposed a privacy-preserving and byzantine-robust FL scheme (P2BroFL) that adopted three-party computation (3PC) to achieve an optimized byzantine-robust aggregation method and local model selection.

The above schemes [20–22,26] utilize two or more servers for implementing robust detection. One of the servers is utilized for perturbing and aggregating gradient parameters while maintaining them in an encrypted state. Another server possesses the decryption key and acquires the perturbed gradient parameters. However, these existing schemes expose the "shifted" distribution of the complete gradient [24]. This is a clear privacy violation [8,25], as it leaks more information, and does not meet the design goal of "privacy" claimed in their works. Furthermore, some schemes [20,21,26,27,35] are designed specifically for IID data, which can result in a significant decrease in accuracy when applied to Non-IID data. The aforementioned challenges serve as the impetus for our endeavor to pursue privacy-preserving and byzantine-robust FL in Non-IID data scenarios. As evidenced by the comparison results presented in Table 1, our proposed ProFL demonstrates superior security and data adaptability when compared to existing byzantine-robust FL schemes.

## 3. Preliminary knowledge

This section first introduces the LHE technique, and then presents the FL framework.

### 3.1. Linearly homomorphic encryption

LHE [36,37] is an algorithm that enables the direct computation of ciphertexts without relying on private keys. Most privacy-preserving FL frameworks rely on the Paillier cryptosystem [36]. The proposed ProFL is developed based on the Paillier cryptosystem to be compatible with most privacy-preserving FL frameworks. The scheme encompasses the following functionalities (KeyGen, Enc, Dec, Eval).

KeyGen($1^k$) → ($pk$, $sk$): The KeyGen function is a probabilistic polynomial-time (PPT) algorithm that generates a pair of public key $pk$ and private key $sk$, given a security parameter $1^k$.

Enc($pk$, $x$) → $[[x]]$: The Enc function encrypts the plaintext $x$ using the HE public key $pk$ and outputs the ciphertext $[[x]]$.

Dec($sk$, $[[x]]$) → $x$: The Dec function decrypts the HE ciphertext $[[x]]$ using the HE private key $sk$ and outputs the plaintext $x$.

Eval($pk$, $[[x_1]]$, $[[x_2]]$, $f_L$) → $x'$: Given two ciphertexts $[[x_1]]$ and $[[x_2]]$ encrypted with the HE public key $pk$ and a linear function $f_L$, the Eval function satisfies Dec($sk$, $x'$)=$f_L(x_1, x_2)$.

The additive and multiplicative homomorphism properties are included in the Paillier cryptosystem. Specifically, considering two ciphertexts $[[x]]$ and $[[y]]$ encrypted with public key $pk$, along with a randomly chosen plaintext $r$, the operations of ciphertext addition homomorphism and ciphertext multiplication homomorphism are supported as follows: 1) $[[x+y]] = [[x]] \cdot [[y]]$; 2) $[[x]]^r = [[r \cdot x]]$.

### 3.2. Federated learning framework

FL is a machine learning approach that involves $N$ clients and a central server. In this part, we introduce the widely used FL algorithm FedAvg. McMahan et al. are the first to propose FL [38] and the vanilla optimization method FedAvg for FL. The objective of distributed optimization in FL is

$$\min_{w} \left\{ f(\boldsymbol{w}) \triangleq \sum_{k=1}^{N} p_k F_k(\boldsymbol{w}) \right\} \tag{1}$$

where $N$ is the number of clients, $p_k$ is the aggregation weight of client $k$ and satisfies $p_k \geq 0$, $\sum_k p_k = 1$. $F_k(\boldsymbol{w})$ represents the objective of local optimization.

In each round of FL, client $k$ updates $\boldsymbol{w}$ on its local data $D_k$ by the loss function $L(\cdot;\cdot)$, where the loss function uses the Stochastic Gradient Descent (SGD) optimization algorithm. The above procedure is formulated as the objective of local optimization $F_k(\boldsymbol{w})$ such that

$$F_k(\boldsymbol{w}) \triangleq \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim D_k} L(\boldsymbol{x}, \boldsymbol{y}; \boldsymbol{w}) \tag{2}$$

where $(\boldsymbol{x}, \boldsymbol{y}) \sim D_k$ represents the local data of client $k$.

The objective of local optimization $F_k(\boldsymbol{w})$ is to minimize the empirical expectation. The global objective $f_k(\boldsymbol{w})$ can be seen as a combined function for multiple clients, and FL aims to minimize it by optimizing all local optimizations.

It is worth noting that since $\Delta\boldsymbol{w}$ is executed locally, its trustworthiness cannot be guaranteed. Therefore, the Byzantine clients may manipulate the value of $\Delta\boldsymbol{w}$, potentially impacting the aggregation process and compromising the entire system.

For privacy-preserving FL, each updated model $\Delta\boldsymbol{w}$ is encrypted as $[[\Delta\boldsymbol{w}]]$ using the public key $pk$ and the aggregation process operates on encrypted updated models to achieve secure computation. In this case, we use LHE to achieve privacy-preserving FL.

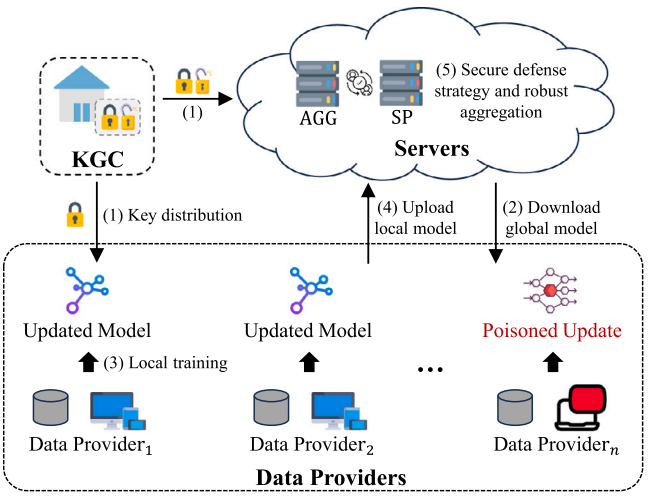



**Fig. 2.** System model of ProFL.

## 4. Problem formulation

In this section, the system model, threat model, and design goals of ProFL are introduced.

### 4.1. System model

As shown in Fig. 2, the system model of ProFL consists of three components: key generation center, $n$ data providers and two servers (Aggregator and Service Provider).

- **Key Generation Center (KGC):** Serving as a trusted authority, the KGC is responsible for distributing and managing the public and private key pairs $(pk, sk)$ within the FL system (Step 1).
- **Data Providers (DPs):** DPs consist of a collection of clients with local data that collaboratively participate in training a FL model under the coordination of servers. In the $t$th training round, $DP_i$ receives a new round of global model parameters (Step 2), updates this model on local data (Step 3), and then uploads the encrypted local model parameters to servers (Step 4).
- **Aggregator (AGG) and Service Provider (SP):** Serving as central servers, AGG and SP interact to implement defense and aggregation against attacks from Byzantine clients (Step 5). After defense and aggregation, a new round of global model parameters is broadcast to DPs (Step 2).

### 4.2. Threat model

KGC is considered to be a fully trusted entity with limited responsibilities restricted to key distribution and management.

Similar to prior works [20,23], DPs are assumed to be categorized into two groups: benign clients and Byzantine clients. Benign clients utilize their local data for model training and upload normal parameters to AGG. Conversely, Byzantine clients engage in both targeted and untargeted attacks and upload poisoned parameters to AGG. Our initial assumption regarding the number of Byzantine clients is as follows

$$|DP^*| \leq \frac{n-1}{2} \tag{3}$$

where $DP^*$ represents Byzantine clients, $n$ represents the number of DPs.

In our framework, AGG only has the LHE public key, which is responsible for performing operations such as aggregating ciphertexts, and does not obtain any plaintext information other than the calculation results. SP has the LHE public–private key pair, which is responsible for decrypting or encrypting ciphertexts. Furthermore, AGG and SP are two honest-but-curious and non-colluding servers that follow the pre-defined protocol, while attempting to extract as much sensitive information as possible from DPs. The non-collusion of the twin-server model has been formalized in prior works [20,22,23]. Considering that numerous companies operating cloud servers are well-established entities, any collusion will inevitably undermine their individual economic interests and tarnish their brand reputations. Based on these assumptions, AGG and SP try to obtain private information about DPs during the execution of the protocols, which constitutes a potential threat.

Furthermore, we explore a threat that overlooked in existing privacy-preserving byzantine-robust FL. The insufficient trustworthiness of Byzantine clients may result in the model parameters being revealed to SP in plaintext. In prior works, SP possess the capability to obtain the perturbed gradient plaintext [20–22,26]. If the SP also obtains the plaintext information of the model parameters used by Byzantine clients, it can derive the perturbation and thus leak the gradient information of other clients [24]. We regard this privacy leakage phenomenon as a new type of security threat.

### 4.3. Design goals

Our objective is to design a byzantine-robust FL scheme in the LHE scenario that can effectively resist attacks launched by Byzantine clients. Additionally, our scheme aims to minimize computational overhead while ensuring security. Considering the threat model, ProFL encompasses three specific design goals.

- **Accuracy.** ProFL should exhibit resilience against both IID and Non-IID data of attacks launched by Byzantine clients. Meanwhile, it should guarantee that the model's global accuracy falls within a reasonable range.
- **Overhead.** With safety as a prerequisite, ProFL necessitates efficient resource allocation coordination among its various components to guarantee optimal system performance and security, especially in the encryption and decryption operations.
- **Privacy.** ProFL must uphold data confidentiality of DPs, refraining from disclosing any DP local data or sensitive information related to local gradients to any server.

## 5. ProFL framework

In this section, we discuss the overall architecture and elaborate on the specific structure of ProFL. The notations implemented are described in Table 2.

### 5.1. ProFL overview

The overall architecture of ProFL is illustrated in Fig. 3. ProFL consists of four phases, namely system init phase (①), local training phase (②), privacy-preserving defense phase (③), and robust aggregation phase (④). The system init phase mainly completes the distribution of public and private key pairs and the initialization of the model parameters. The task of the local training phase is that all DPs update and upload the local encrypted gradients synchronously. In the privacy-preserving defense phase, we design a shuffling-based secure median (**SSecMed**) protocol to securely compute the coordinate-wise median gradient. In the robust aggregation phase, a secure Manhattan distance (**SecMHD**) protocol is proposed to calculate the aggregation weight for each local gradient, and then the local gradients are aggregated and calculated to obtain the global model. The detailed processes are described as follows.

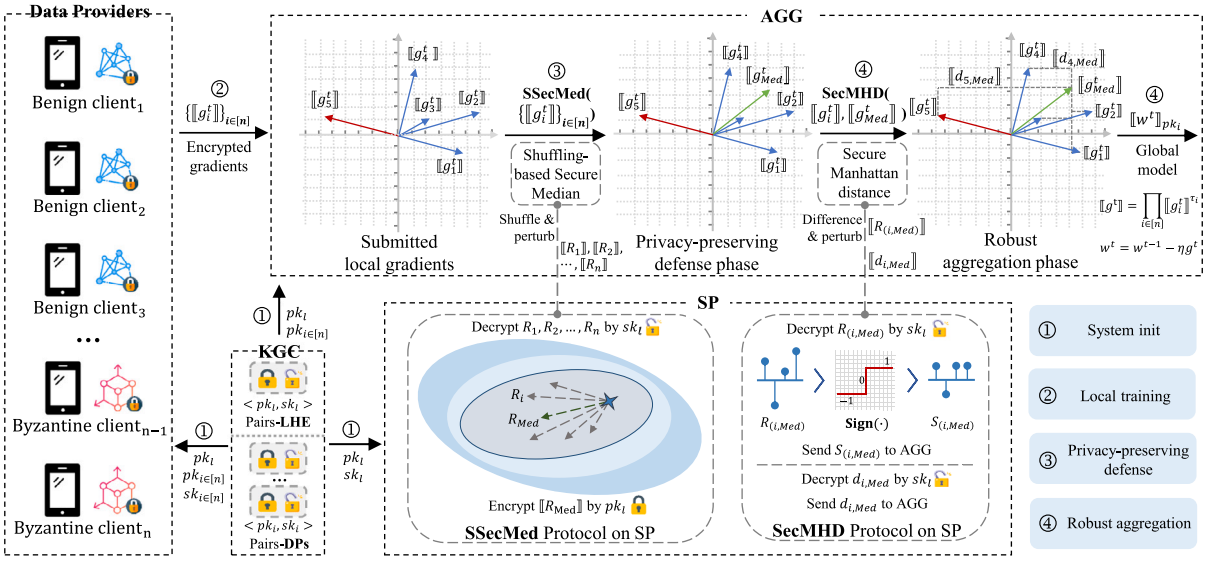



**Fig. 3.** An overview of ProFL.

**Table 2**
Specific notations.

| Notation | Description | Notation | Description |
|---|---|---|---|
| $n$ | Number of DPs | $\boldsymbol{g}_i^t$ | Local gradient |
| $\boldsymbol{w}^t$ | Global model | $m$ | Dimension of gradient |
| $O$ | Shuffled sequence | $\widetilde{\boldsymbol{g}}_i^t$ | Shuffled local gradient |
| $\boldsymbol{R}_i$ | Perturbed gradient | $\boldsymbol{g}_{Med}^t$ | Median gradient |
| $\boldsymbol{D}_{(a,b)}$ | Difference vector | $\boldsymbol{R}_{(a,b)}$ | Perturbed vector |
| $\boldsymbol{S}_{(a,b)}$ | Sign vector | $\mathbf{d}_{a,b}$ | Manhattan distance |
| $\tau_i$ | Aggregation weight | $\eta$ | Learning rate |
| $\kappa$ | Security parameter | $r_j$ | Random perturbation |
| $pk/sk$ | Public/privacy key | $[\![x]\!]$ | Ciphertext of LHE |

### 5.2. Construction of ProFL

#### 5.2.1. System init phase

In ProFL, the KGC is responsible for generating a pair of keys ($pk_l$, $sk_l$) for LHE and distributing the pair of keys to SP, which securely retains the private key $sk_l$. Additionally, the KGC distributes the public key $pk_l$ to all DPs and AGG. In addition, the KGC generates a unique pair of asymmetric keys ($pk_i$, $sk_i$) for each DP to ensure the security of data transmission between AGG and DPs. In the initial phase of ProFL, the parameters of global model $\boldsymbol{w}_0$ are initialized randomly by AGG.

#### 5.2.2. Local training phase

We take the $t$th iteration as an example to elaborate on the local training phase. The specific steps are as follows: in the $t$th iteration, DP$_i$ ($i \in [n]$) receives encrypted model parameters $[[\boldsymbol{w}^{t-1}]]_{pk_i}$ from AGG, and then decrypts and verifies the received model parameters using $sk_i$. Subsequently, DP$_i$ locally trains gradient parameters $\boldsymbol{g}_i^t$ using SGD.

Due to ProFL uses the Paillier cryptosystem, which only supports integer operations, the local gradient is converted into integer form as follows

$$\boldsymbol{g}_i^t = \{\lceil deg \cdot \boldsymbol{g}_{ij}^t \rfloor\}_{j=1}^m \tag{4}$$

where $m$ represents the number of gradient vector parameters, $\lceil\cdot\rfloor \in \mathbb{Z}$ represents the nearest integer to the real number, and $deg$ is the precision degree. $\boldsymbol{g}_{ij}^t$ is the $j$th coordinate entry in the gradient vector $\boldsymbol{g}_i^t$. Finally, DP$_i$ uploads encrypted local gradient $[[\boldsymbol{g}_i^t]]$ to AGG.

#### 5.2.3. Privacy-preserving defense phase

After receiving all local encrypted gradients $[[\boldsymbol{g}_i^t]]_{i\in[n]}$ of the $t$th iteration, ProFL enters the privacy-preserving defense phase. In this phase, ProFL sets the coordinate-wise median gradient $[[\boldsymbol{g}_{Med}^t]]$ as a benchmark for evaluating the quality of local gradients. The coordinate-wise median gradient, which can achieve the optimal order error rate [4] under no more than half of the Byzantine clients. Among them, we design a shuffling-based secure median protocol, abbreviated as **SSecMed** protocol, to securely compute the coordinate-wise median gradient. Specifically, AGG obtains the perturbed vectors $[[\boldsymbol{R}_1]]$, $[[\boldsymbol{R}_2]]$, $\cdots$, $[[\boldsymbol{R}_n]]$ by performing **Shuffle** function and adding perturbations to local encrypted gradients, and sends them to SP. SP decrypts these perturbed vectors $\boldsymbol{R}_1$, $\boldsymbol{R}_2$, $\cdots$, $\boldsymbol{R}_n$ using private key $sk_l$ and calculates the perturbed median vector $\boldsymbol{R}_{Med}$. Subsequently, SP utilizes $pk_l$ to encrypt the vector $[[\boldsymbol{R}_{Med}]]$ and returns it to AGG. The specific process of the privacy-preserving defense phase will be expanded in Section 6.1. The security property of the **SSecMed** protocol will be analyzed in Section 7.1.

#### 5.2.4. Robust aggregation phase

In the robust aggregation phase, a secure Manhattan distance (**SecMHD**) is proposed to calculate the distance $\mathbf{d}_{i,Med}$ between any local gradient $[[\boldsymbol{g}_i^t]]$ and the coordinate-wise median gradient $[[\boldsymbol{g}_{Med}^t]]$ obtained in the privacy-preserving defense phase. These distances $\{\mathbf{d}_{i,Med}\}_{i\in[n]}$ are then weighted inversely as $\{\tau_i\}_{i\in[n]}$ for participation aggregation, rather than discarding suspicious gradients. Specifically, AGG first obtains the perturbed difference vector $[[\boldsymbol{R}_{(i,Med)}]]$ through difference calculation and adding perturbations, which is then transmitted to SP. SP decrypts $\boldsymbol{R}_{(i,Med)}$ with the private key $sk_l$, and subsequently uses the **Sign** function to calculate $\boldsymbol{S}_{(a,b)}$ on the vector $\boldsymbol{R}_{(a,b)}$, which is returned to AGG. The distance $[[\mathbf{d}_{i,Med}]]$ is calculated on AGG in the LHE scenarios, and then it is decrypted through SP to obtain its plaintext. These distances $\{\mathbf{d}_{i,Med}\}_{i\in[n]}$ are inversely weighted as the weights $\{\tau_i\}_{i\in[n]}$ for the aggregation of global gradient. The global

gradient $[[g^t]]$ is obtained from $\prod_{i\in[n]}[[g_i^t]]^{\tau_i}$ and then decrypted by SP. Finally, the global model $\boldsymbol{w}^t$ is calculated from the global model $\boldsymbol{w}^{t-1}$ and the global gradient $\boldsymbol{g}^t$. $[[\boldsymbol{w}^t]]_{pk_i}$ is encrypted by the public key $pk_i$ of $\mathrm{DP}_i$ and sent to $\mathrm{DP}_i$ to complete an iteration of training. The specific process of the robust aggregation phase will be expanded in Section 6.2. The security property of the **SecMHD** protocol will be analyzed in Section 7.1.

## 6. Methodology

In this section, we elaborate in detail on the two key phases within ProFL scheme, namely privacy-preserving defense within ProFL and robust aggregation within ProFL.

### 6.1. Privacy-preserving defense within ProFL

Upon receiving encrypted local gradients $[[g_1^t]]$, $[[g_2^t]]$, ⋯, $[[g_n^t]]$, AGG and SP cooperate to perform privacy-preserving defense phase. In this phase, the benchmark for coordinate-wise median gradient is securely established by **SSecMed**. Next, a comprehensive explanation of **SSecMed** protocol is presented, as depicted in Algorithm 1.

First, AGG uses the **Shuffle** function to randomly shuffle the elements in the coordinate vector sequence $[m]$ to obtain the shuffled sequence $O$, where the **Shuffle** function is implemented based on a Pseudorandom Generator (PRG) [39]. The security of PRG ensures that the output of the generator is indistinguishable from the uniform distribution in polynomial-time, as long as the seed is hidden from the distinguisher. Based on the shuffled sequence $O$, AGG shuffles the gradient vector $[[g_i^t]]$ to obtain the shuffled gradient vector $[[\widetilde{g}_i^t]]$ such that

$$[[\widetilde{g}_{ij}^t]] = [[g_{io}^t]], \text{ for } j, o \in \mathbf{zip}([m], O) \tag{5}$$

where **zip**(*pairs) represents pairing elements from multiple iterable objects in sequence.

Then, AGG randomly selects $m$ nonzero integers $\{r_j\}_{j\in[m]}$ and multiplies them with the $j$th coordinate entity of the $i$th shuffled gradient vector $[[\widetilde{g}_{ij}^t]]$ to generate perturbed vector $[[\boldsymbol{R}_{ij}]]$ such that

$$[[\boldsymbol{R}_{ij}]] = [[\widetilde{g}_{ij}^t]] \cdot [[r_j]], \text{ for } j \in [m] \tag{6}$$

where $r_j \neq 0$.

After that, AGG sends $n$ perturbed vectors $[[\boldsymbol{R}_1]]$, $[[\boldsymbol{R}_2]]$, ⋯, $[[\boldsymbol{R}_n]]$ to SP. SP firstly uses the LHE privacy key $sk_l$ to decrypt these perturbed vectors to obtain the plaintext of perturbed vectors $\{\boldsymbol{R}_i\}_{i\in[n]}$. SP then calculates the coordinate-wise Median $\boldsymbol{R}_{Med}$ of $n$ perturbed vectors. Next, SP uses the LHE public key $pk_l$ to encrypt $\boldsymbol{R}_{Med}$ and obtain the encrypted perturbed Median $[[\boldsymbol{R}_{Med}]]$, which is then sent back to AGG.

After receiving the encrypted perturbed Median, AGG retrieves the shuffled Median gradient $[[\widetilde{g}_{Med}^t]]$ from the encrypted perturbed Median using $r_j$ such that

$$[[\widetilde{g}_{(Med)j}^t]] = [[\boldsymbol{R}_{(Med)j}]] \cdot [[r_j]]^{-1}, \text{ for } j \in [m] \tag{7}$$

Then, AGG uses the shuffled sequence $O$ to restore the original Median gradient $[[g_{Med}^t]]$ based on the shuffled Median gradient $[[\widetilde{g}_{Med}^t]]$ such that

$$[[g_{(Med)o}^t]] = [[\widetilde{g}_{(Med)j}^t]], \text{ for } j, o \in \mathbf{zip}([m], O) \tag{8}$$

Upon the completion of all aforementioned steps, the encrypted Median gradient $[[g_{Med}^t]]$ is returned, thereby concluding the **SSecMed** algorithm.

**Remark.** We take PEFL [20] as an example to analyze how the local gradient parameters of the remaining DPs are revealed when Byzantine clients intentionally expose their gradient parameters. Then, we proceed to analyze how ProFL effectively protects the local gradient parameters of DPs in such scenarios.

**Algorithm 1** Shuffling-based Secure Median (SSecMed)

**Input:** Encrypted local gradients $[[g_1^t]]$, $[[g_2^t]]$, ⋯, $[[g_n^t]]$
**Output:** Encrypted coordinate-wise median gradient $[[g_{Med}^t]]$

AGG:
  /***Shuffle** *represents the function applied to randomly rearrange the elements of an array**/
  $\boldsymbol{O} \leftarrow$ **Shuffle**($[m]$).
  **for** $j, o \in$ **zip**($[m], \boldsymbol{O}$) **do**
    Randomly selects a nonzero integers $r_j$.
    **for** $i \in [n]$ **do**
      $[[\widetilde{g}_{ij}^t]] \leftarrow [[g_{io}^t]]$.
      $[[\boldsymbol{R}_{ij}]] \leftarrow [[\widetilde{g}_{ij}^t]] \cdot [[r_j]]$.
    **end for**
  **end for**
  Sends $[[\boldsymbol{R}_1]]$, $[[\boldsymbol{R}_2]]$, ⋯, $[[\boldsymbol{R}_n]]$ to SP.
SP:
  SP decrypts $n$ ciphertexts of the perturbed gradients:
    $\boldsymbol{R}_i \leftarrow Dec([[\boldsymbol{R}_i]], sk_l)$ *for* $i \in [n]$.
  Computes the median of $n$ plaintexts:
  /***Median** *represents the function applied to calculate the median value of a set of numbers**/
    $\boldsymbol{R}_{Med} \leftarrow$ **Median**($\{\boldsymbol{R}_i\}_{i=1}^n$).
  SP encrypts $\boldsymbol{R}_{Med}$ with $pk_l$:
    $[[\boldsymbol{R}_{Med}]] \leftarrow Enc(\boldsymbol{R}_{Med}, pk_l)$.
  Sends $[[\boldsymbol{R}_{Med}]]$ back to AGG.
AGG:
  **for** $i \in [n]$ **do**
    Remove the perturbation $r_j$:
      $[[\widetilde{g}_{(Med)j}^t]] \leftarrow [[\boldsymbol{R}_{(Med)j}]] \cdot [[r_j]]^{-1}$ *for* $j \in [m]$.
    **for** $j, o \in$ **zip**($[m], \boldsymbol{O}$) **do**
      Restore the shuffled coordinates:
        $[[g_{(Med)o}^t]] \leftarrow [[\widetilde{g}_{(Med)j}^t]]$.
    **end for**
  **end for**
  **return** $[[g_{Med}^t]]$

In the PEFL scheme, the server SP has the ability to decrypt the perturbed vector. For any honest $\mathrm{DP}_i$ and Byzantine DP*, their perturbed parameters of the $j$th coordinate entry are expressed as

$$\begin{cases} g_{ij} + r_j = \boldsymbol{R}_{ij} \\ g_{*j} + r_j = \boldsymbol{R}_{*j} \end{cases} \tag{9}$$

where $g_{ij}$ represents the $j$th coordinate entry in $i$th gradient vector, $g_{*j}$ represents the $j$th coordinate entry in any poisoned gradient vector, $r_j$ is a perturbation for the $j$th coordinate.

If DP* expose its gradient parameters, then the original gradient parameters $g_{*j}$ is obtained by SP. In this case, the original gradient of $\mathrm{DP}_i$ $g_{ij}$ can be derived as follows

$$g_{ij} = (\boldsymbol{R}_{ij} - \boldsymbol{R}_{*j}) + g_{*j} \tag{10}$$

In ProFL scheme, the perturbed parameters of $j$th coordinate entry for any DP are expressed as

$$\begin{cases} g_{ij} + r_j = \boldsymbol{R}_{io} \\ g_{*j} + r_j = \boldsymbol{R}_{*o} \end{cases} \tag{11}$$

In contrast to PEFL, ProFL necessitates that SP possesses the element $o \in O$ on shuffled sequence to accurately calculate $r_j$. Fortunately, given that SP and AGG operate as two non-colluding servers and that the generation of $o \in O$ follows the principles of PRG, it follows that SP is unable to ascertain or deduce the value of $o$. Therefore, ProFL can still effectively protect the gradient information of the remaining DPs in such scenarios.

### 6.2. Robust aggregation within ProFL

During the robust aggregation phase, the Manhattan distance between each local gradient and the coordinate-wise median gradient is computed. Subsequently, an aggregation weight based on this distance is designed, effectively diminishing the influence of poisoned gradients in the aggregation while preserving the contribution of heterogeneous gradients.

The selection of the Manhattan distance is motivated by prior studies [27,29,40] indicating that the Euclidean distance offers a significant advantage in mitigating Byzantine attacks. However, within the Paillier cryptosystem, the computation of Euclidean distance necessitates squaring operations, which are evidently impractical. In contrast, the Manhattan distance not only provides similarity measurement criteria akin to those of the Euclidean distance but also requires merely simple addition operations to yield results. To securely compute the Manhattan distance, a protocol referred to as **SecMHD** is proposed, as detailed in Algorithm 2.

**Algorithm 2** Secure Manhattan distance (SecMHD)

**Input:** Two encrypted local gradients $[[\boldsymbol{g}_a^t]]$, $[[\boldsymbol{g}_b^t]]$
**Output:** Manhattan distance of two gradients $\mathbf{d}_{a,b}$

AGG:
**for** $j \in [m]$ **do**
  Calculate the difference vector between two gradients:
    $[[\boldsymbol{D}_{(a,b)j}]] \leftarrow [[\boldsymbol{g}_{aj}^t]] \cdot [[\boldsymbol{g}_{bj}^t]]^{-1}$
  Randomly select a nonzero integers $r_j$.
  $[[\boldsymbol{R}_{(a,b)j}]] \leftarrow [[\boldsymbol{D}_{(a,b)j}]]^{r_j}$
**end for**
Sends $[[\boldsymbol{R}_{(a,b)}]]$ to SP.
SP:
SP decrypts the perturbed difference vector with $sk_l$:
  $\boldsymbol{R}_{(a,b)} \leftarrow Dec([[\boldsymbol{R}_{(a,b)}]], sk_l)$
**for** $j \in [m]$ **do**
  Computes the sign of the perturbed difference vector:
    /*$\boldsymbol{Sign}$ *represents the function applied to calculate the sign value of the number*/
  $\boldsymbol{S}_{(a,b)j} \leftarrow \mathbf{Sign}(\boldsymbol{R}_{(a,b)j})$
**end for**
Sends $\boldsymbol{S}_{(a,b)}$ back to AGG.
AGG:
Calculate the sum of the difference vector:
  $[[\mathbf{d}_{a,b}]] \leftarrow \prod_{j\in[m]}[[\boldsymbol{D}_{(a,b)j}]]^{\boldsymbol{S}_{(a,b)j}\cdot\mathbf{Sign}(r_j)}$
/*$[[\mathbf{d}_{a,b}]]$ *is decrypted by SP to yield* $\mathbf{d}_{a,b}$*/
**return** $\mathbf{d}_{a,b}$

First, AGG calculates the difference vector $[[\boldsymbol{D}_{(a,b)}]]$ between two gradients $[[\boldsymbol{g}_a^t]]$ and $[[\boldsymbol{g}_b^t]]$ in encrypted state such that

$$[[\boldsymbol{D}_{(a,b)j}]] = [[\boldsymbol{g}_{aj}^t]] \cdot [[\boldsymbol{g}_{bj}^t]]^{-1}, \text{ for } j \in [m] \tag{12}$$

Next, A nonzero random integer $r_j$ is employed to generate a perturbed difference vector $[[\boldsymbol{R}_{(a,b)}]]$ such that

$$[[\boldsymbol{R}_{(a,b)j}]] = [[\boldsymbol{D}_{(a,b)j}]]^{r_j}, \text{ for } j \in [m] \tag{13}$$

where $r_j \neq 0$. The perturbed difference vector $[[\boldsymbol{R}_{(a,b)}]]$ is then sent to SP.

After receiving the perturbed difference vector, SP decrypts it and calculates the sign of vector such that

$$\boldsymbol{S}_{(a,b)j} = \mathbf{Sign}(\boldsymbol{R}_{(a,b)j}), \text{ for } j \in [m] \tag{14}$$

where the **Sign** function is implemented to compute the sign value of the number.

Then, $\boldsymbol{S}_{(a,b)}$ is returned to AGG, which it utilizes $\boldsymbol{S}_{(a,b)}$ and $r_j$ to calculate the sum of the absolute values of the difference vector, which is represented as the Manhattan distance $[[\mathbf{d}_{(a,b)}]]$ between the two gradients $\boldsymbol{g}_a^t$ and $\boldsymbol{g}_a^t$ such that

$$[[\mathbf{d}_{a,b}]] = \prod_{j\in[m]}[[\boldsymbol{D}_{(a,b)j}]]^{\boldsymbol{S}_{(a,b)j}\cdot\mathbf{Sign}(r_j)} \tag{15}$$

Finally, $[[\mathbf{d}_{a,b}]]$ is decrypted by SP using the $sk_l$ and $\mathbf{d}_{a,b}$ is returned.

Based on the **SecMHD** protocol, the Manhattan distance between any local gradient and the coordinate-wise median gradient is calculated and denoted as $\{\mathbf{d}_{i,Med}\}_{i\in[n]}$. A novel aggregation weight $\tau_i$ is designed based on this distance, and it is multiplied by a precision degree $deg$. This is because the Paillier cryptosystem only supports integer arithmetic, so the weights need to be converted to approximate integer forms. The calculation of aggregated weight $\tau_i$ is as follows

$$\tau_i \leftarrow \lceil \frac{\mathbf{d}_{max} - \mathbf{d}_{i,Med}}{n\mathbf{d}_{max} - \sum_{i=1}^{n}\mathbf{d}_{i,Med}} \cdot deg \rfloor \tag{16}$$

where $\mathbf{d}_{max} = \max_{i\in[n]}\mathbf{d}_{i,Med}$, $deg$ is the precision degree, and $\lceil\cdot\rfloor \in \mathbb{Z}$ represents the nearest integer to the real number.

Next, AGG calculates the weighted average of the local gradients as the global gradient $[[\boldsymbol{g}^t]]$ such that

$$[[\boldsymbol{g}^t]] = \prod_{i\in[n]}[[\boldsymbol{g}_i^t]]^{\tau_i} \tag{17}$$

Finally, AGG uploads the encrypted global gradient to SP, which returns $\boldsymbol{g}^t$ to AGG after decrypting by $sk_l$. AGG removes the precision

**Algorithm 3** Framework of ProFL

**Input:** The encrypted local gradients $\{[[\boldsymbol{g}_i^t]]\}_{i\in[n]}$, the learning rate $\eta$, the global rounds $T$, the local rounds $E$, the number of DPs $n$, the local data of $\mathrm{DP}_i$ $D_i$.
**Output:** The global model $\boldsymbol{w}^T$

**for** each communication round $t = 1, 2, ..., T$ **do**
  AGG broadcasts $\{[[\boldsymbol{w}^{t-1}]]_{pk_i}\}_{i\in[n]}$ to all DPs.
  DPs:
  **for** each $\mathrm{DP}_i \in [n]$ synchronously **do**
    $\mathrm{DP}_i$ decrypts and verifies $\boldsymbol{w}^{t-1}$ using $sk_i$.
    $\boldsymbol{g}_i^t \leftarrow LocalUpdating(i, \boldsymbol{w}^{t-1})$
    $\mathrm{DP}_i$ encrypts $\boldsymbol{g}_i^t$ using $pk_l$:
      $[[\boldsymbol{g}_i^t]] \leftarrow Enc(\lceil \boldsymbol{g}_i^t \cdot deg \rfloor, pk_l)$
    $\mathrm{DP}_i$ sends $[[\boldsymbol{g}_i^t]]$ to AGG.
  **end for**
  AGG & SP:
  $[[\boldsymbol{g}_{Med}^t]] \leftarrow \mathbf{SSecMed}([[\boldsymbol{g}_1^t]], [[\boldsymbol{g}_2^t]], \cdots, [[\boldsymbol{g}_n^t]])$
  **for** $i \in [n]$ **do**
    $\mathbf{d}_{\mathbf{i,Med}} \leftarrow \mathbf{SecMHD}([[\boldsymbol{g}_i^t]], [[\boldsymbol{g}_{Med}^t]])$
  **end for**
  **for** $i \in [n]$ **do**
    $\tau_i \leftarrow \lceil \frac{\mathbf{d}_{max} - \mathbf{d}_{i,Med}}{n\mathbf{d}_{max} - \sum_{i=1}^{n}\mathbf{d}_{i,Med}} \cdot deg \rfloor$
    $[[\boldsymbol{g}^t]] \leftarrow [[\boldsymbol{g}^t]] \cdot [[\boldsymbol{g}_i^t]]^{\tau_i}$.
  **end for**
  $[[\boldsymbol{g}^t]]$ is decrypted by SP to yield $\boldsymbol{g}^t$:
    $\boldsymbol{g}^t/deg^2 \leftarrow Dec([[\boldsymbol{g}^t]], sk_l)$
  $\boldsymbol{w}^t \leftarrow \boldsymbol{w}^{t-1} - \eta\boldsymbol{g}^t$
  AGG encrypts $[[\boldsymbol{w}^t]]_{pk_i}$ using $pk_i$ of $\mathrm{DP}_i$.
**end for**
**return** The global model $\boldsymbol{w}^T$
**function** LocalUpdating($i$, $\boldsymbol{w}$)
  **for** each local epoch $e = 1, 2, ..., E$ **do**
    $\boldsymbol{g} \leftarrow \nabla L(D_i; \boldsymbol{w})$
    $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\boldsymbol{g}$
  **end for**
  **return** $\boldsymbol{g}$
**end function**

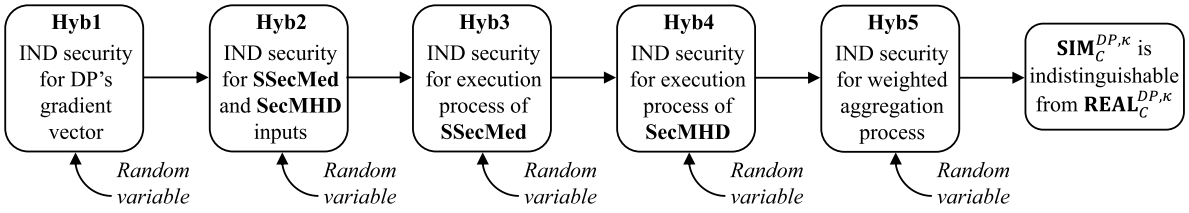



**Fig. 4.** Flowchart of IND-CPA security properties for ProFL.

degree $deg$ such that $\boldsymbol{g}^t = \boldsymbol{g}^t/deg^2$ and updates the global model parameters using the global gradient $\boldsymbol{g}^t$ such that

$$\boldsymbol{w}^t = \boldsymbol{w}^{t-1} - \eta \boldsymbol{g}^t \tag{18}$$

where $\eta$ represents the learning rate. After computation of global model, AGG encrypts the global model $[[\boldsymbol{w}^t]]_{pk_i}$ with public key $pk_i$ of $\mathrm{DP}_i$ and transmits it to $\mathrm{DP}_i$. We end this section by formally summarizing our ProFL in Algorithm 3.

**Remark.** Here, we consider that the privacy of the Manhattan distance $\mathbf{d}_{i,Med}$ does not need be protected. The reason is that even though this distance is public information, it cannot reveal the distribution characteristics of the gradients. This view is consistent with some previous research results, such as the Pearson correlation coefficient [20] and the cosine similarity [22].

## 7. Analysis

In this section, we conduct a comprehensive analysis of the security and convergence of ProFL.

### 7.1. Security analysis

In this section, we provide a hybrid argument that depends on simulators to demonstrate that ProFL leak nothing about the private information of DPs during execution, except for the inferred information about the computation results. We prove that ProFL achieves IND-CPA security [23,36] during protocol execution in the setting of two honest-but-curious and non-colluding servers.

**Proposition 1** (*Honest But Curious And Non-colluding Servers*)**.** *For any DPs and servers C={AGG, SP}, there is a security parameters $\kappa$, let $\mathbf{REAL}_C^{DP,\kappa}$ as a random variable, representing any joint view C parties in reality. Under the condition of $\kappa$ meet security parameters, there is a probabilistic polynomial time (PPT) simulator **SIM**, make the output of the **SIM** and **REAL** are indistinguishable in the calculation:*

$$\mathbf{REAL}_C^{DP,\kappa}(\boldsymbol{g}_i^t, mhd, med) \equiv \mathbf{SIM}_C^{DP,\kappa}(\boldsymbol{g}_i^t, mhd, med) \tag{19}$$

*where $\equiv$ represents computationally indistinguishable.*

The same standard hybrid argument [20,23] is employed to demonstrate that, given a security parameter $\kappa$, under condition of $\kappa$, $\mathbf{SIM}_C^{DP,\kappa}$ is computationally indistinguishable from $\mathbf{REAL}_C^{DP,\kappa}$. As shown in Fig. 4, we encrypt a random variable $\boldsymbol{\psi}_i$ instead of the original gradient vector $\boldsymbol{g}_i^t$, and prove that $\boldsymbol{\psi}_i$ and $\boldsymbol{g}_i^t$ are indistinguishable in the computation (**Hyb1**). Then, under the condition that the previous hybrid holds, we successively prove that the random variables input to **SSecMed** and **SecMHD** (**Hyb2**), the random variables in the execution process of the **SSecMed** protocol (**Hyb3**), the random variables in the execution process of the **SecMHD** protocol (**Hyb4**), and the random variables in the weighted aggregation process (**Hyb5**) are indistinguishable from the original input. The proof is presented as follows.

**Proof.** We initialize a random variable $\mathbf{SIM}_C^{DP,\kappa}$ whose distribution is indistinguishable from the joint view of $C$ parties in the actual execution $\mathbf{REAL}_C^{DP,\kappa}$.

**Hyb1**: In this hybrid, we modify the behavior of DPs, wherein the $\mathrm{DP}_i$ encrypts a randomly vector $\boldsymbol{\psi}_i$ with LHE public key $pk_l$, instead of the original gradient vector $\boldsymbol{g}_i^t$. The calculation of encryption involves only ciphertext operations, and the IND-CPA security property of the LHE [36] ensures that $[[\boldsymbol{\psi}_i]]$ and $[[\boldsymbol{g}_i^t]]$ are indistinguishable in the computation. Therefore, this hybrid is indistinguishable from the previous one.

**Hyb2**: In this hybrid, we change the input of the protocols **SSecMed** and **SecMHD** executed by two non-colluding servers AGG and SP with the randomly vector $[[\boldsymbol{\psi}_i]]$ instead gradient vector $[[\boldsymbol{g}_i^t]]$. The IND-CPA security property of the LHE and non-colluding setting ensures that this hybrid is indistinguishable from the previous one.

**Hyb3**: In this hybrid, we simulate the AGG to compute $[[\boldsymbol{\Theta}_{ij}]] = [[\boldsymbol{\psi}_{io}^t]] \cdot [[\boldsymbol{\zeta}_j]]$ instead of $[[R_{ij}]] = [[\boldsymbol{g}_{io}^t]] \cdot [[r_j]]$, where $j \in [m]$ and $o \in O$. The homomorphic addition operation rules and the setting of two non-colluding servers ensure the indistinguishability of the AGG computation process. Besides, since the noise $\boldsymbol{\zeta}_j$ is sampled uniformly at random and the shuffled sequence $O$ is implemented based on a PRG [39], the parameters $\boldsymbol{\Theta}_i$ obtained after decryption by SP are still a set of random numbers. Therefore, this hybrid is indistinguishable from the previous one.

**Hyb4:** In this hybrid, the calculation of difference vector $[[\boldsymbol{D}_{(a,b)_j}]] = [[\boldsymbol{g}_{aj}^t]] \cdot [[\boldsymbol{g}_{bj}^t]]^{-1}$ can be perfectly simulated by $[[\boldsymbol{\Delta}_{(a,b)_j}]] = [[\boldsymbol{\psi}_{aj}^t]] \cdot [[\boldsymbol{\psi}_{bj}^t]]^{-1}$, where $j \in [m]$. Furthermore, we simulate the AGG to calculate perturbed difference vector $[[\boldsymbol{\Omega}_{(a,b)j}]] = [[\boldsymbol{\Delta}_{(a,b)j}]]^{\zeta_j}$ instead of $[[\boldsymbol{R}_{(a,b)j}]] = [[\boldsymbol{D}_{(a,b)j}]]^{r_j}$, where $j \in [m]$. The homomorphic addition operation rules and the setting of two non-colluding servers ensure the indistinguishability of the AGG computation process. Additionally, similar to Hyb3, since the noise $\boldsymbol{\zeta}_j$ is sampled uniformly at random, the vector $\boldsymbol{\Omega}_{(a,b)}$ obtained by SP is still a set of random numbers. Therefore, this hybrid is indistinguishable from the previous one.

**Hyb5:** In this hybrid, the weighted aggregated gradient is computed by $[[\boldsymbol{g}^t]] = \prod_{i\in[n]}[[\boldsymbol{g}_i^t]]^{\tau_i}$, which can be simulated by $[[\boldsymbol{\psi}^t]] = \prod_{i\in[n]}[[\boldsymbol{\psi}_i^t]]^{\tau_i}$. All operations in the hybrid involve only additive homomorphisms and multiplicative homomorphisms. The IND-CPA security property and two non-colluding servers ensure that this hybrid is indistinguishable from the previous one.

The aforementioned argument demonstrates the existence of a security parameter $\kappa$, such that the simulator $\mathbf{SIM}_C^{DP,\kappa}$ drawn from the aforementioned distribution produces output that is computationally indistinguishable from $\mathbf{REAL}_C^{DP,\kappa}$. Therefore, ProFL has security property, of which curious AGG and SP will have no knowledge of the private data of benign DPs. □

### 7.2. Convergence analysis

In this section, we conduct an analysis of the convergence properties of ProFL.

We assume that the set of clients in the system is $[n]$, where the proportion of Byzantine clients is $\alpha(\alpha < 0.5)$, represented by the set

$\mathcal{B} \subset [n]$, satisfying $|\mathcal{B}| = \alpha n$. Meanwhile, there are $1-\alpha$ proportion of benign clients in the system, whose set is represented as $\mathcal{H} \subset [n]$, where $|\mathcal{H}| = (1-\alpha)n$. Besides, we define $F_k(\cdot)$ as the local loss of client $k$, $F(\cdot)$ as the global loss, and $w$ as the model parameters.

If a client is benign, it can be assumed that it will calculate the local gradient followed the rules, denoted as $\nabla F_k(\boldsymbol{w}_t)$; on the other hand, if a client is Byzantine, it can be assumed that it will upload any gradient, denoted as $*$. In the $t$th iteration, the local gradient of byzantine-robust FL $\boldsymbol{g}_k(\boldsymbol{w}_t)$ can be expressed as

$$\boldsymbol{g}_k(\boldsymbol{w}_t) = \begin{cases} \nabla F_k(\boldsymbol{w}_t), & k \in \mathcal{H} \\ *, & k \in \mathcal{B} \end{cases} \tag{20}$$

We set $\boldsymbol{w}_t$ as the global model parameters for round $t$, and set $\boldsymbol{w}_t^k$ as the local model parameters for client $k$ in round $t$. Each client performs local iterations $E$ times, and when $e = 0$, all clients start updating from the global model parameters $\boldsymbol{w}_t$ from the last round. Therefore, the update for client $k$ in round $t$ is represented as

$$\boldsymbol{w}_{t,e}^k = \begin{cases} \boldsymbol{w}_t, & e = 0 \\ \boldsymbol{w}_{t,e-1}^k - \eta \boldsymbol{g}_k(\boldsymbol{w}_{t,e-1}^k), & e \in [1, E] \end{cases} \tag{21}$$

**Definition 1.** For client $k$, the difference between the local gradient of byzantine-robust FL $\boldsymbol{g}_k(\boldsymbol{w}_t)$ and the benign local gradient $\nabla F_k(\boldsymbol{w}_t)$ is defined as follows

$$\|\boldsymbol{g}_k(\boldsymbol{w}_t) - \nabla F_k(\boldsymbol{w}_t)\| = \epsilon_k \tag{22}$$

where $\epsilon_k = \{0, *\}$. We will continue to discuss the impact of the value of $\epsilon_k$ on convergence in our subsequent analysis.

We make some assumptions about the loss function, which have been proposed in many related works on FL [41,42].

**Assumption 1.** For any client $k$, the local loss $F_k(\cdot)$ is convex

**Assumption 2.** The local loss $F_k(\cdot)$ is M-Lipschitz continuous, which implies that for any $w$ and $v$, it holds that

$$\|F_k(\boldsymbol{w}) - F_k(\boldsymbol{v})\| \leq M\|\boldsymbol{w} - \boldsymbol{v}\| \tag{23}$$

**Assumption 3.** The local loss function $F_k(\cdot)$ is L-Lipschitz smooth, which implies that for any $\boldsymbol{w}$ and $\boldsymbol{v}$, it holds that

$$\|\nabla F_k(\boldsymbol{w}) - \nabla F_k(\boldsymbol{v})\| \leq L\|\boldsymbol{w} - \boldsymbol{v}\| \tag{24}$$

**Definition 2.** For any client $k$, the difference between the benign local gradient and the global gradient is denoted by

$$\delta_k = \max_{\boldsymbol{w}} \|\nabla F_k(\boldsymbol{w}) - \nabla F(\boldsymbol{w})\| \tag{25}$$

and the gradient divergence is defined as

$$\delta \triangleq \sum_{k \in [n]} \tau_k \delta_k \tag{26}$$

where $\tau_k$ represents the weight of $\delta_k$. This weighted divergence has also been employed in previous works [43,44].

To analyze the impact of Non-IID data on the convergence bound, we introduce a virtual model $\boldsymbol{v}_t$ to balance the disparity between FL and centralized training based on SGD. This method has been deployed in many studies on convergence analysis of heterogeneous FL [43–45]. We assume the existence of a virtual model $\boldsymbol{v}_{t,e}$, which performs the same number of rounds as $\boldsymbol{w}_{t,e}$, but differs in that it is centrally trained on the complete data set, representing the convergent properties of the model in an ideal state such that

$$\boldsymbol{v}_{t,e} = \begin{cases} \boldsymbol{w}_t, & e = 0 \\ \boldsymbol{v}_{t,e-1} - \eta \nabla F(\boldsymbol{v}_{t,e-1}), & e \in [1, E] \end{cases} \tag{27}$$

**Proposition 2** (*Upper Bound of the Divergence of $\boldsymbol{w}_{t,e}^k$*). *Under Assumptions 1, 2, and 3 hold, the divergence upper bound of the local model and virtual model of the ProFL for any $t$ and $e$ is*

$$\|\boldsymbol{w}_{t,e}^k - \boldsymbol{v}_{t,e}\| \leq \frac{\delta_k + \epsilon_k}{L}((\eta L + 1)^e - 1) \tag{28}$$

We obtain Eq. (28) by means of Assumption 3, Definition 1 and Definition 2, along with the inequality scaling. The upper bound of the divergence of $\boldsymbol{w}_{t,e}^k$ is proved in detail below.

**Proof.** Based on the smoothness of $F_k(\cdot)$ on Assumption 3 and $\epsilon_k$ of Definition 1 and $\delta_k$ of Definition 2, we have

$$\begin{aligned} &\|\boldsymbol{w}_{t,e}^k - \boldsymbol{v}_{t,e}\| \\ &= \|\boldsymbol{w}_{t,e-1}^k - \eta \boldsymbol{g}_k(\boldsymbol{w}_{t,e-1}^k) - \boldsymbol{v}_{t,e-1} + \eta \nabla F(\boldsymbol{v}_{t,e-1})\| \\ &\leq \|\boldsymbol{w}_{t,e-1}^k - \boldsymbol{v}_{t,e-1}\| + \eta\|\boldsymbol{g}_k(\boldsymbol{w}_{t,e-1}^k) - \nabla F(\boldsymbol{v}_{t,e-1})\| \\ &\leq \|\boldsymbol{w}_{t,e-1}^k - \boldsymbol{v}_{t,e-1}\| + \eta\|\nabla F_k(\boldsymbol{w}_{t,e-1}^k) - \nabla F(\boldsymbol{v}_{t,e-1})\| \\ &\quad + \eta\|\boldsymbol{g}_k(\boldsymbol{w}_{t,e-1}^k) - \nabla F_k(\boldsymbol{w}_{t,e-1}^k)\| \\ &\leq \|\boldsymbol{w}_{t,e-1}^k - \boldsymbol{v}_{t,e-1}\| + \eta\|\nabla F_k(\boldsymbol{w}_{t,e-1}^k) - \nabla F_k(\boldsymbol{v}_{t,e-1})\| \\ &\quad + \eta\|\nabla F_k(\boldsymbol{v}_{t,e-1}) - \nabla F(\boldsymbol{v}_{t,e-1})\| \\ &\quad + \eta\|\boldsymbol{g}_k(\boldsymbol{w}_{t,e-1}^k) - \nabla F_k(\boldsymbol{w}_{t,e-1}^k)\| \\ &\leq \|\boldsymbol{w}_{t,e-1}^k - \boldsymbol{v}_{t,e-1}\| + \eta L\|\boldsymbol{w}_{t,e-1}^k - \boldsymbol{v}_{t,e-1}\| + \eta(\delta_k + \epsilon_k) \end{aligned} \tag{29}$$

Let $h(e) = \|\boldsymbol{w}_{t,e}^k - \boldsymbol{v}_{t,e}\|$, then above equation can be rewritten as

$$h(e) \leq (\eta L + 1)h(e-1) + \eta(\delta_k + \epsilon_k) \tag{30}$$

By adding $(\delta_k + \epsilon_k)/L$ to both sides of the inequality, it can be simplified to

$$\frac{h(e) + (\delta_k + \epsilon_k)/L}{h(e-1) + (\delta_k + \epsilon_k)/L} \leq \eta L + 1 \tag{31}$$

where $h(0) = \|\boldsymbol{w}_{t,0}^k - \boldsymbol{v}_{t,0}\| = 0$, then above equation can be derived as

$$h(e) + \frac{\delta_k + \epsilon_k}{L} \leq \frac{\delta_k + \epsilon_k}{L}(\eta L + 1)^e \tag{32}$$

Therefore, Proposition 2 is proved. □

Prior works have shown that the coordinate-wise Median gradient has the optimal order error rate [4,30], so we define the difference between the coordinate-wise Median gradient and any gradient of benign client as follows.

**Definition 3.** For any client $k$, the maximum difference between the benign local gradient and the coordinate-wise Median gradient is denoted by

$$\delta_{Med} = \max_{w} \|\nabla F_k(\boldsymbol{w}) - \nabla F_{Med}(\boldsymbol{w})\| \tag{33}$$

**Proposition 3** (*Convergence Bound of ProFL Under the Proportion of $\alpha n$ Byzantine Clients*). *Let Assumption 1 to 4 and Definitions 1 to 3 hold, under $g$, $t$, $E$, $\boldsymbol{w}_t$, $\boldsymbol{v}_{t,e}$ be defined therein. If $\tau_i$ is treated as the weight for aggregation, then the convergence bound between the federated model and the virtual model is $\frac{(\eta L+1)^E - 1}{L}(\delta + \alpha n \frac{\delta_{Med}{}^2}{4})M$.*

Based on Proposition 2, with the aid of Definition 2, Definition 3 and inequality scaling, we derive the convergence bound between the federated model $F(\boldsymbol{w}_t)$ and the vitual model $F(\boldsymbol{v}_{t,e})$. The convergence bound of ProFL under the proportion of $\alpha n$ Byzantine clients is proved in detail below.

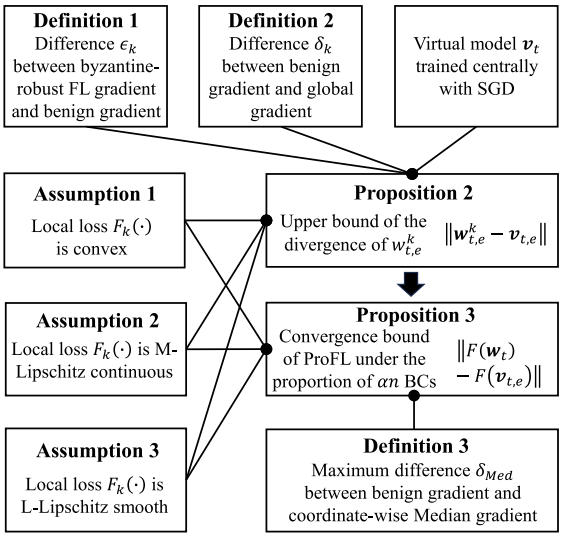



**Fig. 5.** Flowchart of convergence analysis for ProFL.

**Proof.** Deduced from Definition 2 and Proposition 2, we have

$$\begin{aligned}
&\|\boldsymbol{w}_t - \boldsymbol{v}_{t,e}\| \\
&= \sum_{k\in[n]} \tau_k \|\boldsymbol{w}_{t,E}^k - \boldsymbol{v}_{t,e}\| \\
&\le \sum_{k\in[n]} \tau_k \frac{\delta_k + \epsilon_k}{L}((\eta L+1)^E - 1) \\
&= \frac{(\eta L+1)^E - 1}{L}\Big[\sum_{h\in\mathcal{H}} \tau_h \delta_h + \sum_{b\in\mathcal{B}} \tau_b(\delta_b + \epsilon_b)\Big] \\
&= \frac{(\eta L+1)^E - 1}{L}\Big[\sum_{k\in[n]} \tau_k \delta_k + \sum_{b\in\mathcal{B}} \tau_b \epsilon_b\Big] \\
&= \frac{(\eta L+1)^E - 1}{L}\Big[\delta + \sum_{b\in\mathcal{B}} \tau_b \epsilon_b\Big]
\end{aligned} \tag{34}$$

where $\epsilon_h = 0$ and $\sum_{k\in[n]} \tau_k \delta_k = \delta$.

Based on Eq. (16), it can be derive that the weight $\tau_b$ is inversely proportional to the distance between its gradient $\boldsymbol{g}_b(\boldsymbol{w})$ and the coordinate-wise median gradient $\boldsymbol{g}_{Med}(\boldsymbol{w})$. Combing Definition 3 and using Jensen's inequality we get

$$\begin{aligned}
&\tau_b = -\|\boldsymbol{g}_b(\boldsymbol{w}) - \nabla F_{Med}(\boldsymbol{w})\| \\
&\le \|\nabla F_b(\boldsymbol{w}) - \nabla F_{Med}(\boldsymbol{w})\| - \|\boldsymbol{g}_b(\boldsymbol{w}) - \nabla F_b(\boldsymbol{w})\| \\
&\le \delta_{Med} - \epsilon_b
\end{aligned} \tag{35}$$

Therefore, $\sum_{b\in\mathcal{B}} \tau_b \epsilon_b$ can be derived below

$$\begin{aligned}
\sum_{b\in\mathcal{B}} \tau_b \epsilon_b &\le \sum_{b\in\mathcal{B}} (\delta_{Med} - \epsilon_b)\epsilon_b \\
&\le \alpha n \frac{{\delta_{Med}}^2}{4}
\end{aligned} \tag{36}$$

Substituting the above equation into Eq. (34) yields

$$\|\boldsymbol{w}_t - \boldsymbol{v}_{t,e}\| \le \frac{(\eta L+1)^E - 1}{L}(\delta + \alpha n \frac{{\delta_{Med}}^2}{4}) \tag{37}$$

Consider the continuous of $F(\cdot)$ we have

$$\|F(\boldsymbol{w}_t) - F(\boldsymbol{v}_{t,e})\| \le \frac{(\eta L+1)^E - 1}{L}(\delta + \alpha n \frac{{\delta_{Med}}^2}{4})M \tag{38}$$

Therefore, Proposition 3 is proved. □

In conclusion, the assumptions, definitions, and propositions proposed in Section 7.2, as well as the execution flow among them are shown in Fig. 5.

## 8. Experimental analysis

We utilize an Ubuntu 18.04 system equipped with Pytorch, running on an Intel(R) Xeon(R) Gold 6326 CPU at 2.90 GHz, an NVIDIA A10 Tensor Core GPU, and 64 GB of RAM. Leveraging the open-source FL framework *PFLlib* [46], we establish several environments, incorporating a variety of Byzantine clients. In terms of encryption, some open-source homomorphic encryption frameworks [47,48] are adopted in the experiments. We again employ *joblib* to parallel the operations here. The code is available at https://github.com/zzzzzzp02/ProFL.

### 8.1. Experimental setup

#### 8.1.1. Datasets, data settings, model architectures

In this subsection, the introduction of datasets, data settings, and model architectures used in our experiments is provided.

**(a) Dataset Selection:** We evaluate the performance of ProFL on three benchmark datasets: MNIST, Fashion-MNIST (FMNIST), and CIFAR10. MNIST comprises 60,000 samples of handwritten digits, each represented as a 28 × 28 pixel grayscale image, categorized into 10 classes. FMNIST consists of 60,000 samples of clothing images, also 28 × 28 pixels in grayscale, classified into 10 different categories. CIFAR10 includes 60,000 samples of object images, each being a 32 × 32 pixel color image, divided into 10 categories. For MNIST and FMNIST datasets, images are converted to tensors and normalized using a mean of 0.5 and a standard deviation of 0.5 for the single grayscale channel. For CIFAR10, images are converted to tensors and normalized for each RGB channel using a mean and standard deviation of 0.5.

**(b) Data Partitioning:** We conduct experiments across various Non-IID and IID settings. The Dirichlet distribution [49] is employed to allocate training samples among the DPs.

**(c) Model Architectures:** All datasets are trained using an identical CNN model with a consistent architecture, comprising two convolutional layers with kernel sizes of 5 × 5, and two fully connected layers, each containing 512 neurons.

#### 8.1.2. Byzantine clients in ProFL

Two representative types of attacks launched by Byzantine clients are considered: targeted attacks and untargeted attacks. **Targeted Attacks**: We instruct the Byzantine clients to convert the source label $s$ of its local sample to the target label $n-s-1$, where $n$ is the total number of labels in the dataset. **Untargeted Attacks**: The Byzantine clients multiply the updated model parameter $w^b$ with a negative random number $c$, denoted as $c \cdot w^b$, where $c < 0$. In our experiments, we set $c = -1$.

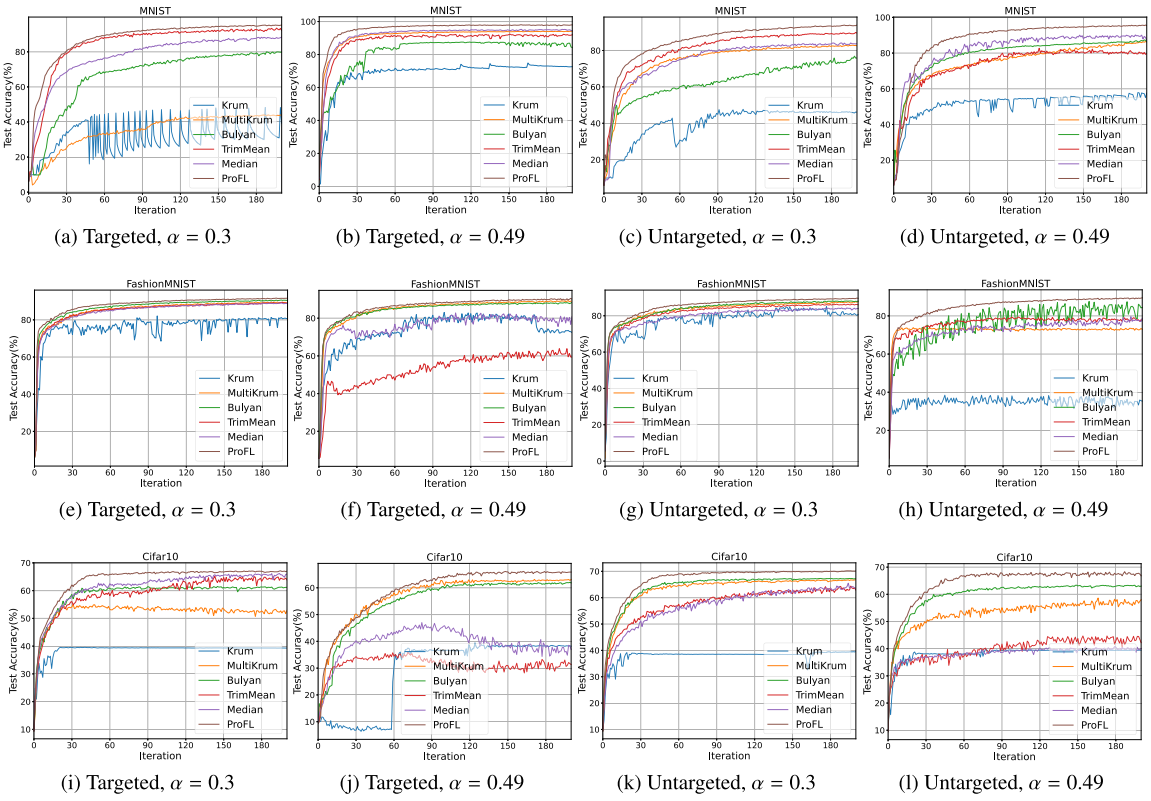
#### 8.1.3. Other schemes for comparison

We primarily compare ProFL with two categories of schemes. The first category comprises classical byzantine-robust FL schemes that have been widely recognized for their effectiveness, such as Krum [29], Multi-Krum [29], Bulyan [28], Median [30], and Trimmed Mean [30]. The second category consists of the latest privacy-preserving and byzantine-robust or heterogeneous FL algorithms, including PEFL [20] (TIFS 2021), ShieldFL [22] (TIFS 2022), SEAR [27] (TDSC 2022), Byzantine-robust Variance-reduced FL (VRFL) [33] (Information Sciences 2022), and P2BroFL [26] (TDSC 2023).

### 8.2. Experimental results

#### 8.2.1. Performance on different proportion of Byzantine clients

We conduct accuracy experiments with proportions of Byzantine clients including $\alpha = \{0.3, 0.49\}$ on the MNIST, FMNIST, and CIFAR10 datasets, respectively. These three datasets are sampled by clients using the Dirichlet distribution, with sampling proportions of 0.5. The following parameters are utilized for the training related to FL: the learning rate is set to 0.005, the batch size is set to 50, the local iteration




(a) Targeted, $\alpha = 0.3$ (b) Targeted, $\alpha = 0.49$ (c) Untargeted, $\alpha = 0.3$ (d) Untargeted, $\alpha = 0.49$

(e) Targeted, $\alpha = 0.3$ (f) Targeted, $\alpha = 0.49$ (g) Untargeted, $\alpha = 0.3$ (h) Untargeted, $\alpha = 0.49$

(i) Targeted, $\alpha = 0.3$ (j) Targeted, $\alpha = 0.49$ (k) Untargeted, $\alpha = 0.3$ (l) Untargeted, $\alpha = 0.49$

**Fig. 6.** Performance with different proportion of Byzantine clients on three datasets.

is set to 1, the global iteration is set to 200, and the number of clients is set to 20.

Fig. 6 (6(a)–6(d)) show the performance of ProFL on the MNIST dataset under targeted attacks and untargeted attacks. It is evident that the accuracy of ProFL against the two types of attacks can be maintained at above 93%. Fig. 6 (6(e)–6(h)) show that the performance of ProFL on the FMNIST dataset under targeted attacks and untargeted attacks is such that the accuracy of ProFL against the two types of attacks can be maintained at above 89%. Fig. 6 (6(i)–6(l)) show the performance of ProFL on the CIFAR10 dataset under targeted attacks and untargeted attacks, and it is clear that the accuracy of ProFL against the two types of attacks can be maintained at above 66%. During the training process of various datasets and attack types, Krum algorithm shows significant fluctuations and a relatively low accuracy. We speculate that this is because it selects only one optimal local gradient as the global gradient in each round and discards the other local gradients. In the case of Non-IID data, this processing method discards a considerable amount of heterogeneous knowledge, thereby making the trained model struggle to adapt to the overall data distribution. Similar phenomena are also presented in algorithms such as MultiKrum, TrimMean, etc., where these algorithms discard a part of the parameters when performing the defense. Conversely, ProFL assigns corresponding aggregation weights to distinct local gradients rather than directly eliminating the suspicious gradients, which performs better than each algorithm in different datasets and different attack types. This performance highlights the significant advantage of our scheme in mitigating attacks over other schemes.

#### 8.2.2. *Performance on different Dirichlet distributed data*

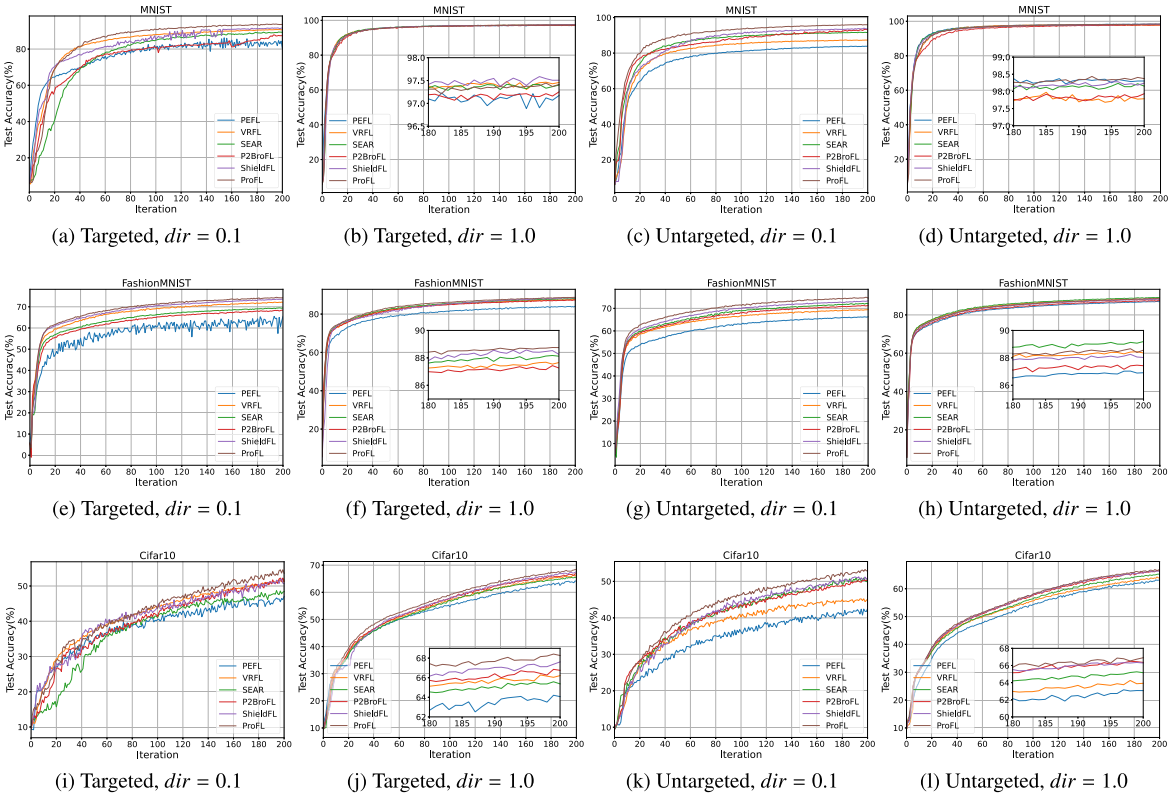
In this section, we conduct experiments on the datasets of MNIST, FMNIST, and CIFAR10. The proportion of Byzantine clients is set to $\alpha = 0.49$, targeted and untargeted attacks are considered by the experiments separately, and the Dirichlet distributions are specified as $dir = \{0.1, 0.4, 0.7, 1.0\}$. For all experiment settings, the global iteration is set to 200, the local iteration is set to 1, the batch size is set to 50, the learning rate is set to 0.005. Our ProFL scheme is compared with the latest privacy-preserving or heterogeneous byzantine-robust FL schemes, including PEFL [20], ShieldFL [22], SEAR [27], VRFL [33], and P2BroFL [26].

Table 3 shows the performance of ProFL under different Dirichlet distributions on the MNIST, FMNIST, and CIFAR10 datasets, respectively. From the results of the three datasets, we observe that when $dir = 1.0$, taking two attacks of the MNIST dataset as examples, all algorithms can achieve similar accuracy. We speculate that the reason lies in the fact that the similarity between benign gradients at $dir = 1.0$ is closer, which makes it easier to identify benign gradients. Moreover, as the Non-IID degree increases, that is, as the parameter $dir$ decreases, the accuracy of all algorithms decreases. Among them, PEFL performs the worst, and we suspect that the reason is that the direct filtering of suspicious gradients may mistakenly delete some benign but highly deviant gradients in the Non-IID setting. Unlike these solutions, ShieldFL and ProFL adopt a weighting strategy rather than direct filtering, and demonstrate superior performance in high Non-IID environments. This result indicates that our ProFL demonstrates greater adaptability and robustness in heterogeneous environments.

Furthermore, we compare the details of accuracy of various byzantine-robust FL algorithms (such as PEFL, VRFL, SEAR, P2BroFL, and ShieldFL as shown in Section 8.1.3) under Dirichlet distributions of $dir = \{0.1, 1.0\}$, and the results are presented in Fig. 7. From the experimental results of the three datasets, most of the existing algorithms can resist two types of poisoning attacks when $dir = 1.0$. Especially on the MNIST dataset, the details of the accuracy of basically all algorithms are similar. However, the performance of different algorithms begins to show differences when $dir = 0.1$. For example, PEFL and VRFL exhibit significant fluctuations and accuracy drops on the Cifar10 dataset. On the contrary, our ProFL performs best on all datasets when $dir = 0.1$, which reflects the superiority of our ProFL in the Non-IID data setting.

**Table 3**
Performance with different Dirichlet distributed data on three benchmark datasets.

| Datasets | Methods | Targeted attack | | | | Untargeted attack | | | |
|---|---|---|---|---|---|---|---|---|---|
| | | $dir = 0.1$ | $dir = 0.4$ | $dir = 0.7$ | $dir = 1.0$ | $dir = 0.1$ | $dir = 0.4$ | $dir = 0.7$ | $dir = 1.0$ |
| MNIST | PEFL [20] | 0.8622 | 0.8736 | 0.9523 | 0.9722 | 0.8396 | 0.8932 | 0.9820 | 0.9837 |
| | VRFL [33] | 0.9077 | 0.9192 | 0.9509 | 0.9748 | 0.8738 | 0.9469 | 0.9804 | 0.9803 |
| | SEAR [27] | 0.8942 | 0.9274 | 0.9557 | 0.9741 | 0.9176 | 0.9448 | 0.9835 | 0.9823 |
| | P2BroFL [26] | 0.8780 | 0.9281 | 0.9544 | 0.9725 | 0.9332 | 0.9361 | 0.9866 | 0.9793 |
| | ShieldFL [22] | 0.9165 | 0.9401 | 0.9589 | 0.9758 | 0.9406 | 0.9548 | 0.9874 | 0.9828 |
| | ProFL | 0.9383 | 0.9592 | 0.9603 | 0.9744 | 0.9607 | 0.9712 | 0.9867 | 0.9843 |
| FMNIST | PEFL [20] | 0.6548 | 0.8274 | 0.8394 | 0.8415 | 0.6628 | 0.8351 | 0.848 | 0.8703 |
| | VRFL [33] | 0.7222 | 0.8411 | 0.8628 | 0.8768 | 0.6946 | 0.8574 | 0.8769 | 0.8845 |
| | SEAR [27] | 0.6949 | 0.833 | 0.8745 | 0.8817 | 0.7222 | 0.8554 | 0.8502 | 0.8919 |
| | P2BroFL [26] | 0.6838 | 0.8483 | 0.8724 | 0.8743 | 0.7123 | 0.8742 | 0.8731 | 0.8747 |
| | ShieldFL [22] | 0.7363 | 0.8571 | 0.8823 | 0.8857 | 0.7334 | 0.8799 | 0.8783 | 0.8825 |
| | ProFL | 0.7446 | 0.8742 | 0.8897 | 0.8875 | 0.7491 | 0.881 | 0.8855 | 0.8867 |
| Cifar10 | PEFL [20] | 0.4705 | 0.5279 | 0.6279 | 0.6421 | 0.4252 | 0.5151 | 0.6209 | 0.6316 |
| | VRFL [33] | 0.5181 | 0.5559 | 0.6579 | 0.6628 | 0.4522 | 0.5479 | 0.6379 | 0.6424 |
| | SEAR [27] | 0.4869 | 0.536 | 0.6241 | 0.6558 | 0.5097 | 0.5316 | 0.6434 | 0.6525 |
| | P2BroFL [26] | 0.5226 | 0.5383 | 0.6324 | 0.6685 | 0.5030 | 0.5271 | 0.6367 | 0.6648 |
| | ShieldFL [22] | 0.5197 | 0.5671 | 0.6623 | 0.6761 | 0.5126 | 0.558 | 0.6533 | 0.6635 |
| | ProFL | 0.5461 | 0.5872 | 0.6603 | 0.6841 | 0.5325 | 0.5732 | 0.6597 | 0.6691 |




(a) Targeted, $dir = 0.1$ (b) Targeted, $dir = 1.0$ (c) Untargeted, $dir = 0.1$ (d) Untargeted, $dir = 1.0$

(e) Targeted, $dir = 0.1$ (f) Targeted, $dir = 1.0$ (g) Untargeted, $dir = 0.1$ (h) Untargeted, $dir = 1.0$

(i) Targeted, $dir = 0.1$ (j) Targeted, $dir = 1.0$ (k) Untargeted, $dir = 0.1$ (l) Untargeted, $dir = 1.0$

**Fig. 7.** Performance with Dirichlet distribution of $dir = \{0.1, 1.0\}$ on three benchmark datasets.

*8.2.3. Complexity analysis and cost experiment on ProFL*

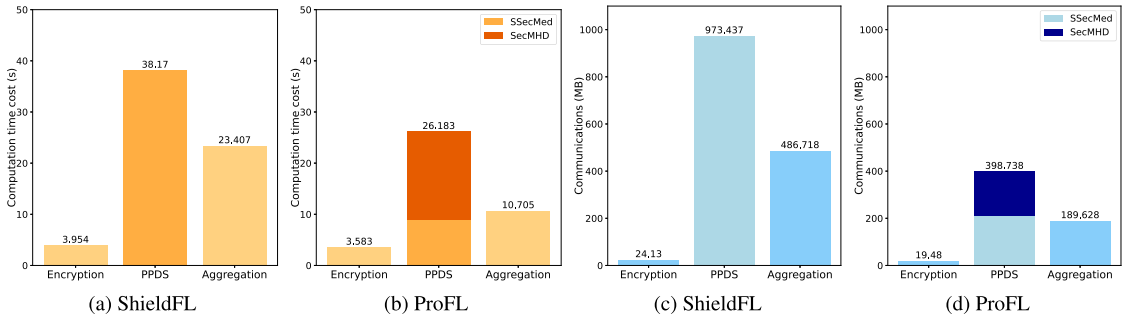
In this part, we conduct a complexity analysis and cost experiment on the protocols of each part on ProFL. To quantify the impact of LHE on complexity, we formalize each atomic step involved in these operations [22] including encryption ($T_{Enc}$), decryption ($T_{Dec}$), homomorphic multiplication ($T_{Mul}$), and homomorphic addition ($T_{Add}$). Let $|X|$ be the communication complexity of an encrypted number.

During the local training phase, each local gradient $\boldsymbol{g}_i^t$ has a shape of $1 \times m$, and the encryption complexity required for each gradient is $\mathcal{O}(mT_{Enc})$ and the communication complexity of $\mathcal{O}(m|X|)$. During the execution of the protocol **SSecMed**, AGG needs to perform $m$ homomorphic additions for each of the $n$ clients, with the computation complexity of $\mathcal{O}(nmT_{Add})$ and the communication complexity of $\mathcal{O}(nm|X|)$. Additionally, SP needs to decrypt these parameters and encrypt the newly generated parameters, with the computation complexity of $\mathcal{O}(nmT_{Dec} + mT_{Enc})$ and the communication complexity of $\mathcal{O}(m|X|)$. In the execution of the protocol **SecMHD**, AGG needs to perform two homomorphic additions and two homomorphic multiplications on $m$-dimensional gradient vector, with the computation complexity of $\mathcal{O}(2m(T_{Add} + T_{Mul}))$. Then, AGG transmits the perturbed vector with a communication complexity of $\mathcal{O}(m|X|)$. SP then needs to decrypt the ciphertext, with the computation complexity of $\mathcal{O}(mT_{Dec})$. In the aggregation phase, $n$ gradients are multiplied and added using homomorphic additions and multiplications, with the computation complexity of $\mathcal{O}(nm(T_{Mul}+T_{Add}))$ and the communication complexity of $\mathcal{O}(m|X|)$. The aggregated gradient needs to be decrypted by SP, with the

**Table 4**
Computation and communication complexity on ProFL.

| Phase | ProFL | |
|---|---|---|
| | Compu. | Comm. |
| Local (@[$n$]) | $\mathcal{O}(mT_{Enc})$ | $\mathcal{O}(m\|X\|)$ |
| SSecMed | $\mathcal{O}(nm(T_{Add}+T_{Dec})+mT_{Enc})$ | $\mathcal{O}((n+1)m\|X\|)$ |
| SecMHD (@[$n$]) | $\mathcal{O}(2m(T_{Add}+T_{Mul})+mT_{Dec})$ | $\mathcal{O}(m\|X\|)$ |
| Aggregation | $\mathcal{O}(nm(T_{Add}+T_{Mul})+mT_{Dec})$ | $\mathcal{O}(m\|X\|)$ |

*Notes:* The abbreviation for communication complexity is "Comm.", and the abbreviation for computation complexity is "Compu."




(a) ShieldFL (b) ProFL (c) ShieldFL (d) ProFL

**Fig. 8.** Computation time cost and communication cost of each components of ShieldFL and ProFL algorithms in one iteration. *Encryption*: Cost of client-side encryption for local gradients. *PPDS (Privacy-preserving Defense Strategy)*: Cost of privacy-preserving defense strategy, among which in ProFL refer to the cost required for the execution of the **SSecMed** and **SecMHD** protocols. *Aggregation*: Cost of server-side global aggregation for gradients.

computation complexity of $\mathcal{O}(mT_{Dec})$. The complexity summary of the protocols of each part in ProFL is presented in Table 4. In comparison with other schemes such as PEFL [20] and ShieldFL [22], as we have explained in Section 7.1, Eq. (9), and Eq. (11), ProFL offers higher security than other schemes. In addition, ProFL can still maintain the same linear growth in computation and communication complexity with respect to the number of clients and parameters as other schemes.

Next, we compare the computation time cost and communication cost of each component of ShieldFL and ProFL in one iteration, as shown in Fig. 8. The number of clients and the model architecture used for training remain consistent from Section 8.1. We record the computation time cost and communication cost related to the ciphertext. The computation time cost is measured in seconds (s), while the communication cost is measured in megabytes (MB). The experimental results show that in one iteration, compared with ShieldFL, ProFL reduces the computation time cost by 38.23% and the communication cost by 58.98% respectively. We analyze that this is because ProFL employs an efficient linearly homomorphic cryptosystem, while ShieldFL uses a two-trapdoor homomorphic system and needs to perform multiplicative homomorphism on two ciphertexts, resulting in a higher cost. During the implementation of the ProFL scheme, the cost for the clients is significantly smaller than that for the servers, which ensures that the client can operate efficiently in some scenarios with limited resources.

Furthermore, we utilize the simulation proposed in [48] and the iteration profiling results described in Section 8.2.2 to predict the total computation time cost and communication cost for convergence. We record the epochs needed for the algorithm to reach the best convergence, the accuracy rate, as well as the computation time cost (in hours, h) and the communication cost (in Gigabytes, GB). Three algorithms are set in this part, namely ShieldFL, ProFL (Cipher.), and ProFL (Plain.). ProFL (Cipher.) represents the ProFL algorithm in the form of ciphertext and ProFL (Plain.) represents the ProFL algorithm in the form of plaintext. The experimental results are shown in Table 5. From the experimental results of convergence achieved in the three datasets, ProFL (Plain.) requires no encryption and hence achieves the fastest possible training convergence. Compared with the ShieldFL algorithm, ProFL (Cipher.) shortens the convergence training time for MNIST, FMNIST, and Cifar10 by 1.62×, 1.78×, and 2.56×, respectively. Meanwhile, the communication costs are reduced by about 2.41× on three datasets. We stress that the ProFL (Cipher.) adopts precision encoding $deg$, and therefore these performance improvements are achieved without reducing the quality of the training model [20]. In conclusion, ProFL (Cipher.) demonstrates a much lower cost than ShieldFL in terms of computation time cost and communication cost across all datasets while maintaining comparable accuracy with ProFL (Plain.), making it a more efficient and secure choice.

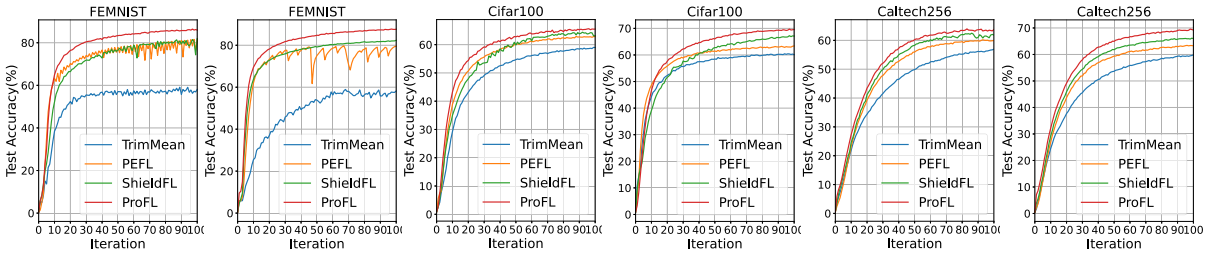
#### 8.2.4. Extended experiments on complex data scenarios

In this part, we use FEMNIST [50], Cifar100 [51], and CalTech256 [52] as benchmark datasets. The training model used for the FEMNIST dataset is the CNN model architecture described in Section 8.1, while the training models used for the Cifar100 and CalTech256 datasets are a pre-trained ResNet18 model, whose parameters are obtained through pre-training on the ImageNet dataset and are used as the initial local models. The size of each image in the FEMNIST dataset is adjusted to a grayscale image of 28 × 28 pixels. The images are converted to tensors and normalized with a mean and standard deviation of 0.5. The size of each image in the Cifar100 dataset and the CalTech256 dataset is adjusted to 128 × 128 pixels, and a central 112 × 112 area is cropped from them. The images are converted to tensors and normalized in the Cifar100 dataset. The mean used for normalization is [0.485, 0.456, 0.406], and the standard deviation is [0.229, 0.224, 0.225].

The Dirichlet distribution with $dir = 0.5$ is conducted to divide the original dataset. The global rounds are set to 100, and 20% of Byzantine clients are introduced to participate in the training process. The other parameters of FL are executed in accordance with the standards stipulated in Section 8.1. To significantly evaluate the performance, we compare the ProFL with the three schemes of TrimMean [30], PEFL [20], and ShieldFL [22]. Among them, TrimMean is a classic byzantine-robust FL scheme and is representative to a certain extent. PEFL is a typical FL scheme that takes both privacy-preserving and byzantine-robustness into account. ShieldFL is especially optimized for privacy-preserving and byzantine-robust FL in the Non-IID data setting. The experimental results are shown in Fig. 9. TrimMean and PEFL suffer significant losses and obvious fluctuations in training accuracy. The speculated reason is that TrimMean and PEFL adopt a direct filtering method when defending against suspicious parameters, while ShieldFL and ProFL apply a confidence weighting strategy, making them more adaptable when dealing with Non-IID data. Furthermore, the training process of the Cifar100 and CalTech256 datasets is more

**Table 5**
Projected total computation time cost and communication cost until convergence for the three datasets. *ProFL (Cipher.)*: The ProFL algorithm in the form of ciphertext. *ProFL (Plain.)*: The ProFL algorithm in the form of plaintext.

| Dataset | Mode | Epochs | Accuracy | Times (h) | Traffic (GB) |
|---|---|---|---|---|---|
| MNIST | ShieldFL | 68 | 98.15% | 5.87 | 101.14 |
| | ProFL (Cipher.) | 70 | 98.37% | 3.62 | 41.96 |
| | ProFL (Plain.) | 70 | 98.37% | 1.11 | 6.76 |
| FMNIST | ShieldFL | 131 | 85.64% | 11.08 | 188.78 |
| | ProFL (Cipher.) | 127 | 86.96% | 6.23 | 78.53 |
| | ProFL (Plain.) | 127 | 86.96% | 1.47 | 13.22 |
| Cifar10 | ShieldFL | 183 | 65.51% | 49.68 | 263.74 |
| | ProFL (Cipher.) | 176 | 66.24% | 19.42 | 109.71 |
| | ProFL (Plain.) | 176 | 66.24% | 4.16 | 18.31 |




(a) Targeted (b) Untargeted (c) Targeted (d) Untargeted (e) Targeted (f) Untargeted

**Fig. 9.** Performance with different schemes on three benchmark datasets (FEMNIST, Cifar100, and CalTech256).

stable compared to that of the FEMNIST dataset. We speculate that the training model architectures of these two datasets are based on pre-trained parameters rather than being initialized from a completely random state. This makes the attack range of Byzantine clients relatively limited, thereby making the training process more stable. Overall, ProFL demonstrates outstanding performance in three complex datasets and two types of poisoning attack scenarios, highlighting its strong adaptability in dealing with complex data scenarios.

#### 8.2.5. Extended experiments on advanced attack scenarios

In this part, we conduct a comparative analysis of the robustness of various schemes in defending against various advanced attacks (including LIE attack [53], Fang's attack [4], MinMax attack [6], and FMPA [54]). Specifically, Fang's attack adopts the specific method of Fang's attack on Krum; for the MinMax attack, its deviation direction is set as the unit vector opposite to the ideal direction. We refer to the implementation methods of poisoning attacks proposed by the *MODEL* [55]. It should be noted that some advanced attack methods, such as Fang's attack and MinMax attack, assume that Byzantine clients can obtain key information such as the local models of all clients or the robust aggregation method adopted by the server. However, such assumptions are essentially in conflict with the original design intention of secure FL (Section 4.3). Even then, studying the robustness of ProFL in defending against poisoning attacks under both full-knowledge assumption and semi-knowledge assumption [6] is still one of the key indicators for evaluating its capabilities.

We follow the experimental setup described in Section 8.2.1 and carefully select several representative schemes, including TrimMean [30], PEFL [20], and ShieldFL [22]. The results obtained from the experiments and the details of the training process are presented in Table 6 and Fig. 10, respectively. The experimental results show that among the above four attack scenarios, ProFL performs the best. In specific attack scenarios such as LIE attack or Fang's attack, the performance of PEFL and ShieldFL is even lower than that of TrimMean. We speculate that PEFL sets a filtering threshold, and these attacks can cleverly remain within the threshold range, thereby evading its detection mechanism. ShieldFL selects the local gradient with the largest difference from the global gradient in the previous round as the benchmark. However, the design goal of these attacks is commonly to maintain within a reasonable gradient range as the optimization focus, which makes it difficult for ShieldFL to identify the true poisoned gradient. In contrast, ProFL effectively narrows the poisoning range of the poisoned gradient and enhances the adaptability to Non-IID data by selecting the parameters with the optimal order error rate [30] as the benign benchmark and evaluating the confidence of each local gradient accordingly.

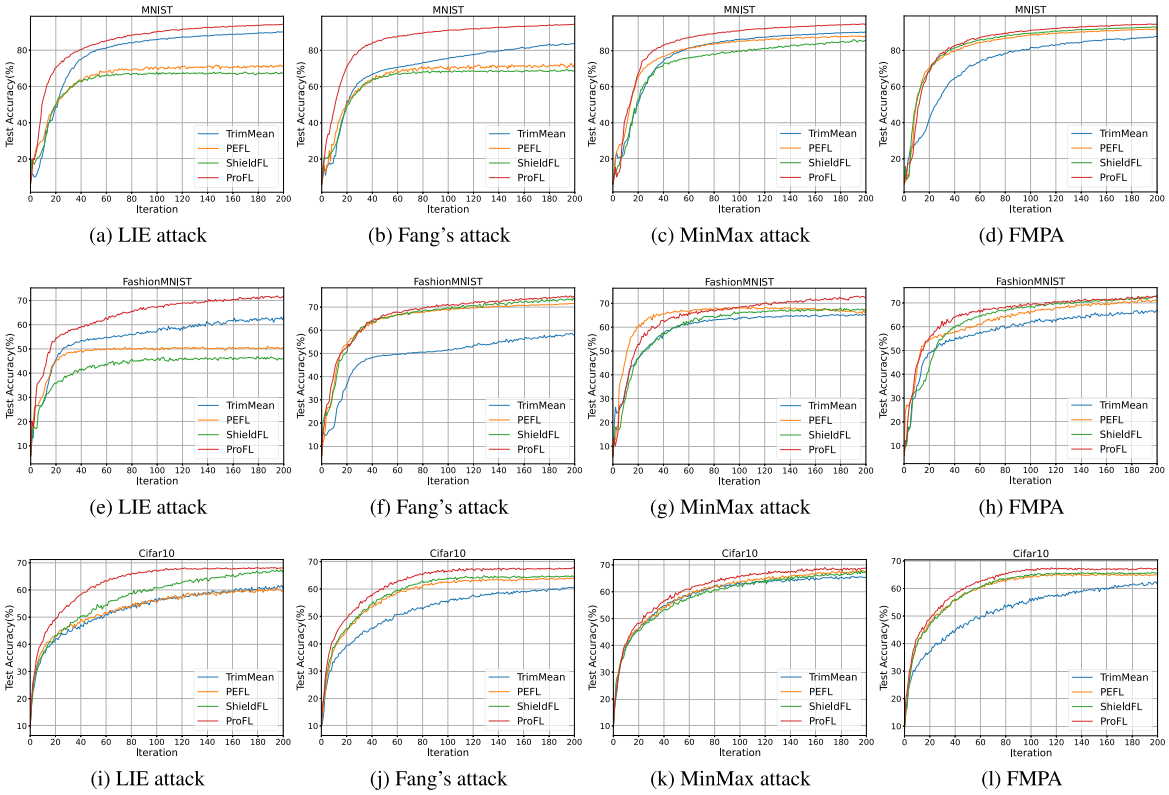
## 9. Discussion

**Strengths & weaknesses:** The proposed ProFL scheme addresses a potential privacy leakage in current privacy-preserving and byzantine-robust FL [20–23], which is the exposure of the "shifted" distribution of the complete gradient [8,24]. Such leakage may cause the local gradient parameters to be exposed to honest but curious servers. Moreover, given the complexity and computational overhead of homomorphic encryption (HE), we merely implement the secure calculation of the benchmark model and the gradient similarity using the linearly HE cryptosystem. Finally, in contrast to other byzantine-robust FL schemes [20,21,26], ProFL does not directly discard the abnormal gradients. Instead, it assigns distinct aggregation weights to different gradients based on their similarity to the benchmark model, facilitating its application in the Non-IID data setting. To the best of our knowledge, ProFL is the first scheme that calculates the similarity of two gradients in the linearly HE cryptosystem without exposing the "shifted" distribution. Although some studies [12,56] have shown that HE with additive fields can significantly reduce the computational cost and such technologies have been applied in high-performance devices or mobile devices, yet in some resource-constrained scenarios, the introduction of linearly HE still imposes a considerable burden on the devices. This may lead to a reduction in the scalability of ProFL in these scenarios.

**Potential applications:** The proposed ProFL offers an effective solution that considers both privacy-preserving and robustness for FL training. For instance, during the training process of disease diagnosis models in the medical field [57], as well as that of risk assessment and credit scoring models in the financial field [58], ProFL can guarantee that these data are processed on siloes or devices and only encrypted gradients are uploaded to the server, thereby preventing the direct

**Table 6**
Performance with four poisoning attack types (LIE attack, Fang's attack, MinMax attack, and FMPA) on three benchmark datasets.

| Datasets | Methods | Poisoning attack types | | | |
|---|---|---|---|---|---|
| | | LIE [53] | Fang's [4] | MinMax [6] | FMPA [54] |
| MNIST | TrimMean [30] | 0.9021 | 0.8377 | 0.9033 | 0.8781 |
| | PEFL [20] | 0.7185 | 0.7291 | 0.8823 | 0.9188 |
| | ShieldFL [22] | 0.6791 | 0.6932 | 0.8612 | 0.9301 |
| | ProFL | 0.9427 | 0.9441 | 0.9479 | 0.9464 |
| FMNIST | TrimMean [30] | 0.6311 | 0.5875 | 0.6533 | 0.6681 |
| | PEFL [20] | 0.5110 | 0.7158 | 0.6844 | 0.7190 |
| | ShieldFL [22] | 0.4682 | 0.7375 | 0.6797 | 0.7278 |
| | ProFL | 0.7188 | 0.7485 | 0.7301 | 0.7305 |
| Cifar10 | TrimMean [30] | 0.6165 | 0.6065 | 0.6567 | 0.6249 |
| | PEFL [20] | 0.6072 | 0.6402 | 0.6795 | 0.6537 |
| | ShieldFL [22] | 0.6738 | 0.6487 | 0.6793 | 0.6586 |
| | ProFL | 0.6825 | 0.6783 | 0.6901 | 0.6751 |




(a) LIE attack (b) Fang's attack (c) MinMax attack (d) FMPA

(e) LIE attack (f) Fang's attack (g) MinMax attack (h) FMPA

(i) LIE attack (j) Fang's attack (k) MinMax attack (l) FMPA

**Fig. 10.** Performance with different types of poisoning attacks on three benchmark datasets.

leakage of privacy data. Meanwhile, its distinctive mechanism can defend against abnormal circumstances such as data contamination and poisoning, enhance the robustness of the FL training process and improve the accuracy and reliability of the model.

## 10. Conclusion

This paper proposes a privacy-preserving and byzantine-robust FL scheme called ProFL to identify Byzantine clients under stringent privacy protection. ProFL designs two secure protocols to mitigate potential threats in existing methods and proposes a robust aggregation method to enhance the robustness of federated aggregation in Non-IID data scenarios. Comprehensive theoretical analysis has proved the security and convergence of the ProFL scheme. The experimental results show that ProFL has considerable accuracy and robustness on multiple benchmark datasets and poisoning attacks. In future work, we will focus on improving the ability of ProFL in defending against various advanced poisoning attacks and further explore ways to optimize the efficiency.

## CRediT authorship contribution statement

**Zongpu Wei:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **Jinsong Wang:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Conceptualization. **Zening Zhao:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization. **Zhao Zhao:** Writing – review & editing, Supervision, Methodology.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

The work was supported by the National Natural Science Foundation of China (Grant No. 62072336), the Key Research and Development Program of Tianjin (Grant No. 23YFZCSN00240), the Tianjin Technical Innovation Guidance Special Project (Grant No. 22YDPYGX 00040).

## Data availability

I would like to refer the reader to the following link to access the implementation of the paper. Link: https://github.com/zzzzzzp02/ProFL (github.com).

## References

[1] A. Imteaj, U. Thakker, S. Wang, J. Li, M.H. Amini, A survey on federated learning for resource-constrained IoT devices, IEEE Internet Things J. 9 (1) (2022) 1–24.
[2] Y. Lu, X. Huang, Y. Dai, S. Maharjan, Y. Zhang, Blockchain and federated learning for privacy-preserved data sharing in industrial IoT, IEEE Trans. Ind. Inform. 16 (6) (2020) 4177–4186.
[3] S. Niknam, H.S. Dhillon, J.H. Reed, Federated learning for wireless communications: Motivation, opportunities, and challenges, IEEE Commun. Mag. 58 (6) (2020) 46–51.
[4] M. Fang, X. Cao, J. Jia, N.Z. Gong, Local model poisoning attacks to byzantine-robust federated learning, in: Proceedings of the 29th USENIX Conference on Security Symposium, USENIX Security, Boston, MA, USA, 2020, pp. 1623–1640.
[5] A.N. Bhagoji, S. Chakraborty, P. Mittal, S. Calo, Analyzing federated learning through an adversarial lens, in: International Conference on Machine Learning, ICML, Lugano, Switzerland, 2019, pp. 634–643.
[6] V. Shejwalkar, A. Houmansadr, Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning, in: Network and Distributed System Security Symposium, NDSS, virtual, 2021.
[7] L. Li, W. Xu, T. Chen, G.B. Giannakis, Q. Ling, RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets, in: Proceedings of the AAAI Conference on Artificial Intelligence, AAAI, Honolulu, HI, USA, 2019, pp. 1544–1551.
[8] J. Geiping, H. Bauermeister, H. Dröge, M. Moeller, Inverting gradients - how easy is it to break privacy in federated learning? in: Conference on Neural Information Processing Systems, NeurIPS, virtual, 2020, pp. 16937–16947.
[9] Y. Zhang, R. Jia, H. Pei, W. Wang, B. Li, D. Song, The secret revealer: Generative model-inversion attacks against deep neural networks, in: IEEE/CVF Computer Vision and Pattern Recognition Conference, CVPR, Seattle, WA, USA, 2020, pp. 253–261.
[10] M. Nasr, R. Shokri, A. Houmansadr, Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning, in: IEEE Symposium on Security and Privacy, SP, San Francisco, CA, USA, 2019, pp. 739–753.
[11] D.C. Nguyen, Q.-V. Pham, P.N. Pathirana, M. Ding, A. Seneviratne, Z. Lin, O. Dobre, W.-J. Hwang, Federated learning for smart healthcare: A survey, ACM Comput. Surv. ( Csur) 55 (3) (2022) 1–37.
[12] L.T. Phong, Y. Aono, T. Hayashi, L. Wang, S. Moriai, Privacy-preserving deep learning via additively homomorphic encryption, IEEE Trans. Inf. Forensics Secur. 13 (5) (2018) 1333–1345.
[13] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, R.H. Deng, Privacy-preserving federated deep learning with irregular users, IEEE Trans. Dependable Secur. Comput. 19 (2) (2022) 1364–1381.
[14] D. Byrd, A. Polychroniadou, Differentially private secure multi-party computation for federated learning in financial applications, in: The First ACM International Conference on AI in Finance, ICAIF, New York, NY, USA, 2020, pp. 1–9.
[15] W. Mou, C. Fu, Y. Lei, C. Hu, A verifiable federated learning scheme based on secure multi-party computation, in: International Conference on Wireless Algorithms, Systems, and Applications, WASA, Nanjing, China, 2021, pp. 198–209.
[16] A. Mondal, Y. More, R.H. Rooparaghunath, D. Gupta, Poster: FLATEE: Federated learning across trusted execution environments, in: IEEE European Symposium on Security and Privacy, EuroS&P, Vienna, Austria, 2021, pp. 707–709.
[17] Y. Zhang, Z. Wang, J. Cao, R. Hou, D. Meng, ShuffleFL: gradient-preserving federated learning using trusted execution environment, in: Proceedings of the 18th ACM International Conference on Computing Frontiers, CF, virtual, 2021, pp. 161–168.
[18] Y. Zhao, J. Zhao, M. Yang, T. Wang, N. Wang, L. Lyu, D. Niyato, K.-Y. Lam, Local differential privacy-based federated learning for Internet of Things, IEEE Internet Things J. 8 (11) (2021) 8836–8853.
[19] K. Wei, J. Li, C. Ma, M. Ding, W. Chen, J. Wu, M. Tao, H.V. Poor, Personalized federated learning with differential privacy and convergence guarantee, IEEE Trans. Inf. Forensics Secur. 18 (2023) 4488–4503.
[20] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, R. Lu, Privacy-enhanced federated learning against poisoning adversaries, IEEE Trans. Inf. Forensics Secur. 16 (2021) 4574–4588.
[21] Y. Miao, Z. Liu, H. Li, K.-K.R. Choo, R.H. Deng, Privacy-preserving Byzantine-robust federated learning via blockchain systems, IEEE Trans. Inf. Forensics Secur. 17 (2022) 2848–2861.
[22] Z. Ma, J. Ma, Y. Miao, Y. Li, R.H. Deng, ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning, IEEE Trans. Inf. Forensics Secur. 17 (2022) 1639–1654.
[23] X. Chen, H. Yu, X. Jia, X. Yu, APFed: Anti-poisoning attacks in privacy-preserving heterogeneous federated learning, IEEE Trans. Inf. Forensics Secur. 18 (2023) 5749–5761.
[24] T. Schneider, A. Suresh, H. Yalame, Comments on "privacy-enhanced federated learning against poisoning adversaries", IEEE Trans. Inf. Forensics Secur. 18 (2023) 1407–1409.
[25] B. Hitaj, G. Ateniese, F. Perez-Cruz, Deep models under the GAN: information leakage from collaborative deep learning, in: ACM Conference on Computer and Communications Security, CCS, Dallas, TX, USA, 2017, pp. 603–618.
[26] C. Dong, J. Weng, M. Li, J.-N. Liu, Z. Liu, Y. Cheng, S. Yu, Privacy-preserving and Byzantine-robust federated learning, IEEE Trans. Dependable Secur. Comput. 21 (2) (2024) 889–904.
[27] L. Zhao, J. Jiang, B. Feng, Q. Wang, C. Shen, Q. Li, Sear: Secure and efficient aggregation for byzantine-robust federated learning, IEEE Trans. Dependable Secur. Comput. 19 (5) (2021) 3329–3342.
[28] E.M. El Mhamdi, R. Guerraoui, S. Rouault, The hidden vulnerability of distributed learning in byzantium, in: International Conference on Machine Learning, ICML, Stockholm, Sweden, 2018, pp. 3521–3530.
[29] P. Blanchard, E.M. El Mhamdi, R. Guerraoui, J. Stainer, Machine learning with adversaries: Byzantine tolerant gradient descent, NeurIPS, in: Conference on Neural Information Processing Systems, vol. 30, Long Beach, CA, USA, 2017.
[30] Y. Chen, L. Su, J. Xu, Distributed statistical machine learning in adversarial settings: Byzantine gradient descent, Proc. ACM Meas. Anal. Comput. Syst. 1 (2) (2017) 1–25.
[31] X. Cao, M. Fang, J. Liu, N.Z. Gong, FLTrust: Byzantine-robust federated learning via trust bootstrapping, in: Network and Distributed System Security Symposium, NDSS 2021, virtual, 2021.
[32] H. Xu, T. Shu, Defending against model poisoning attack in federated learning: A variance-minimization approach, J. Inf. Secur. Appl. 82 (2024) 103744.
[33] J. Peng, Z. Wu, Q. Ling, T. Chen, Byzantine-robust variance-reduced federated learning over distributed non-iid data, Inform. Sci. 616 (2022) 367–391.
[34] K. Zhai, Q. Ren, J. Wang, C. Yan, Byzantine-robust federated learning via credibility assessment on non-IID data, 2021, arXiv preprint arXiv:2109.02396.
[35] X. Ma, X. Sun, Y. Wu, Z. Liu, X. Chen, C. Dong, Differentially private byzantine-robust federated learning, IEEE Trans. Parallel Distrib. Syst. 33 (12) (2022) 3690–3701.
[36] P. Paillier, Public-key cryptosystems based on composite degree residuosity classes, in: International Conference on the Theory and Applications of Cryptographic Techniques, EUROCRYPT, Prague, Czechia, 1999, pp. 223–238.
[37] C. Gentry, A. Sahai, B. Waters, Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based, in: Advances in Cryptology-CRYPTO 2013: 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, 2013, pp. 75–92.
[38] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
[39] A.C. Yao, Theory and application of trapdoor functions, in: 23rd Annual Symposium on Foundations of Computer Science, SFCS, Chicago, Illinois, USA, 1982, pp. 80–91.
[40] J. So, B. Güler, A.S. Avestimehr, Byzantine-resilient secure federated learning, IEEE J. Sel. Areas Commun. 39 (7) (2020) 2168–2181.
[41] L. Liu, J. Zhang, S. Song, K.B. Letaief, Client-edge-cloud hierarchical federated learning, in: ICC 2020-2020 IEEE International Conference on Communications, ICC, IEEE, Dublin, Ireland, 2020, pp. 1–6.
[42] S. Wang, T. Tuor, T. Salonidis, K.K. Leung, C. Makaya, T. He, K. Chan, Adaptive federated learning in resource constrained edge computing systems, IEEE J. Sel. Areas Commun. 37 (6) (2019) 1205–1221.
[43] M. Duan, D. Liu, X. Ji, Y. Wu, L. Liang, X. Chen, Y. Tan, A. Ren, Flexible clustered federated learning for client-level data distribution shift, IEEE Trans. Parallel Distrib. Syst. 33 (11) (2021) 2661–2674.
[44] X. Li, K. Huang, W. Yang, S. Wang, Z. Zhang, On the convergence of FedAvg on non-IID data, in: International Conference on Learning Representations, ICLR, New Orleans, Louisiana, USA, 2019.
[45] S.U. Stich, Local SGD converges fast and communicates little, in: International Conference on Learning Representations, ICLR, New Orleans, Louisiana, USA, 2019.
[46] J. Zhang, Y. Liu, Y. Hua, H. Wang, T. Song, Z. Xue, R. Ma, J. Cao, PFLlib: A Beginner-Friendly and comprehensive personalized federated learning library and benchmark, J. Mach. Learn. Res. 26 (50) (2025) 1–10.

[47] C. Data61, Python Paillier library, 2013, https://github.com/data61/python-paillier.
[48] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, Y. Liu, BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning, in: Proceedings of the 2020 USENIX Annual Technical Conference, USENIX ATC, virtual, 2020, pp. 493–506.
[49] T. Lin, L. Kong, S.U. Stich, M. Jaggi, Ensemble distillation for robust model fusion in federated learning, NeurIPS, in: Conference on Neural Information Processing Systems, vol. 33, Vancouver, BC, Canada, 2020, pp. 2351–2363.
[50] S. Caldas, S.M.K. Duddu, P. Wu, T. Li, J. Konečnỳ, H.B. McMahan, V. Smith, A. Talwalkar, Leaf: A benchmark for federated settings, in: Conference on Neural Information Processing Systems, NeurIPS, 2018, pp. 1–9.
[51] A. Krizhevsky, G. Hinton, et al., Learning Multiple Layers of Features from Tiny Images, Tech. Rep., University of Toronto, 2009.
[52] G. Griffin, A. Holub, P. Perona, Caltech-256 Object Category Dataset, Tech. Rep. 7694, California Institute of Technology, 2007.
[53] G. Baruch, M. Baruch, Y. Goldberg, A little is enough: Circumventing defenses for distributed learning, NeurIPS, in: Advances in Neural Information Processing Systems, vol. 32, Vancouver, BC, Canada, 2019.
[54] H. Zhang, Z. Yao, L.Y. Zhang, S. Hu, C. Chen, A. Liew, Z. Li, Denial-of-service or fine-grained control: Towards flexible model poisoning attacks on federated learning, in: Proceedings of the International Joint Conference on Artificial Intelligence, IJCAI, Macao, China, 2023, pp. 4567–4575.
[55] M. Wu, B. Zhao, Y. Xiao, C. Deng, Y. Liu, X. Liu, MODEL: A model poisoning defense framework for federated learning via truth discovery, IEEE Trans. Inf. Forensics Secur. 19 (2024) 8747–8759.
[56] Y. Cao, J. Zhang, Y. Zhao, P. Su, H. Huang, SRFL: A secure & robust federated learning framework for IoT with trusted execution environments, Expert Syst. Appl. 239 (2024) 122410.
[57] C. Zhang, X. Meng, Q. Liu, S. Wu, L. Wang, H. Ning, FedBrain: A robust multi-site brain network analysis framework based on federated learning for brain disease diagnosis, Neurocomputing 559 (2023) 126791.
[58] Z. Jovanovic, Z. Hou, K. Biswas, V. Muthukkumarasamy, Robust integration of blockchain and explainable federated learning for automated credit scoring, Comput. Netw. 243 (2024) 110303.